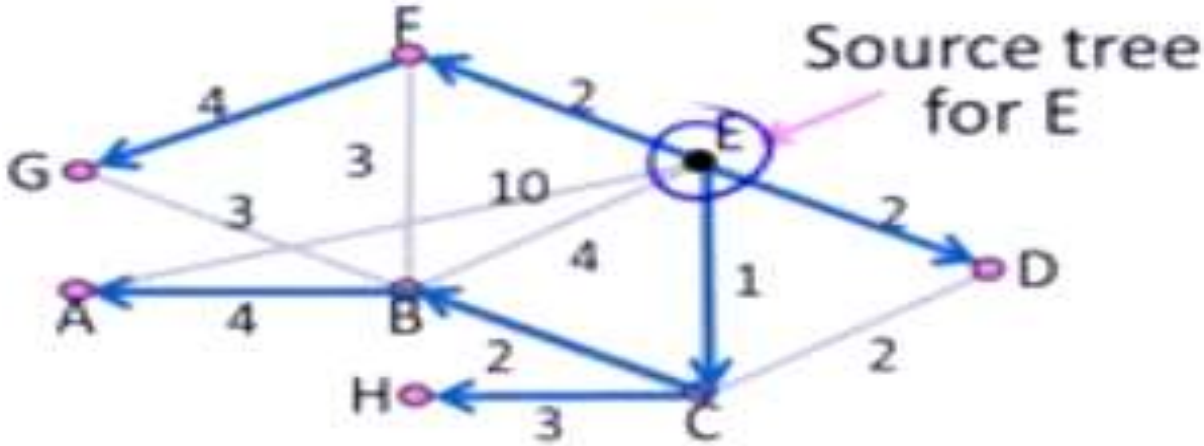
Topic

- How to compute shortest paths given the network topology
  - With Dijkstra's algorithm



Edsger W. Dijkstra (1930-2002)

- Famous computer scientist
  - Programming languages
  - Distributed algorithms
  - Program verification
- Dijkstra's algorithm, 1969
  - Single-source shortest paths, given network with non-negative link costs

By Hamilton Richards, CC-BY-SA-3.0, via Wikimedia Commons

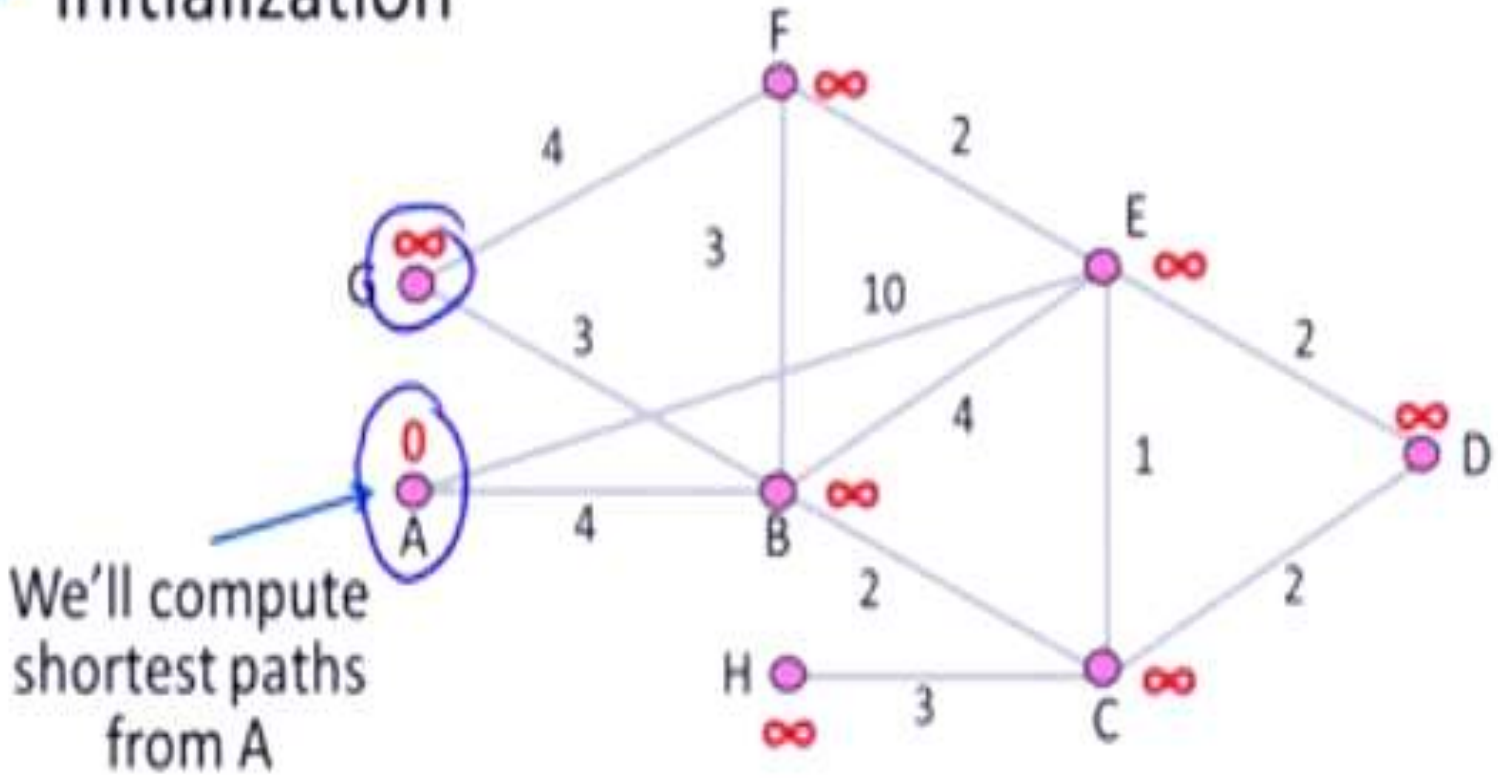
Dijkstra's Algorithm

Algorithm:

- Mark all nodes tentative, set distances from source to 0 (zero) for source, and ∞ (infinity) for all other nodes
- While tentative nodes remain:
  - Extract N, a node with lowest distance
  - Add link to N to the shortest path tree
  - Relax the distances of neighbors of N by lowering any better distance estimates

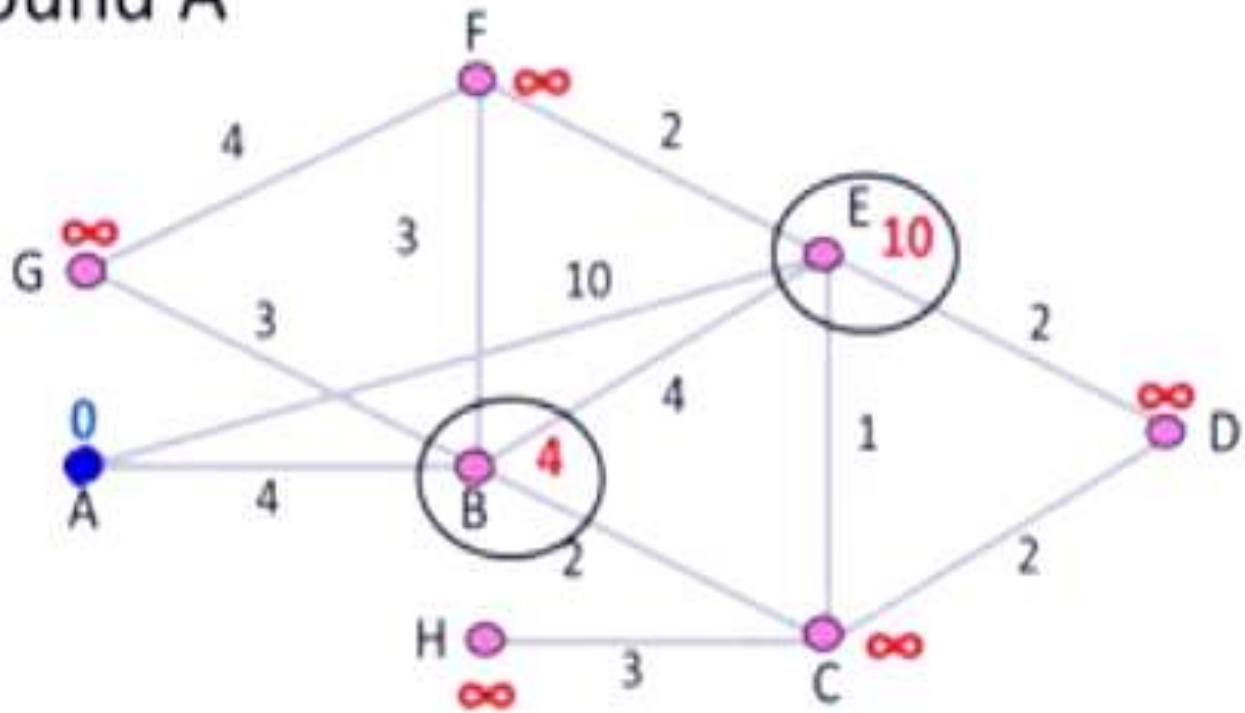
Dijkstra's Algorithm (2)

- Initialization



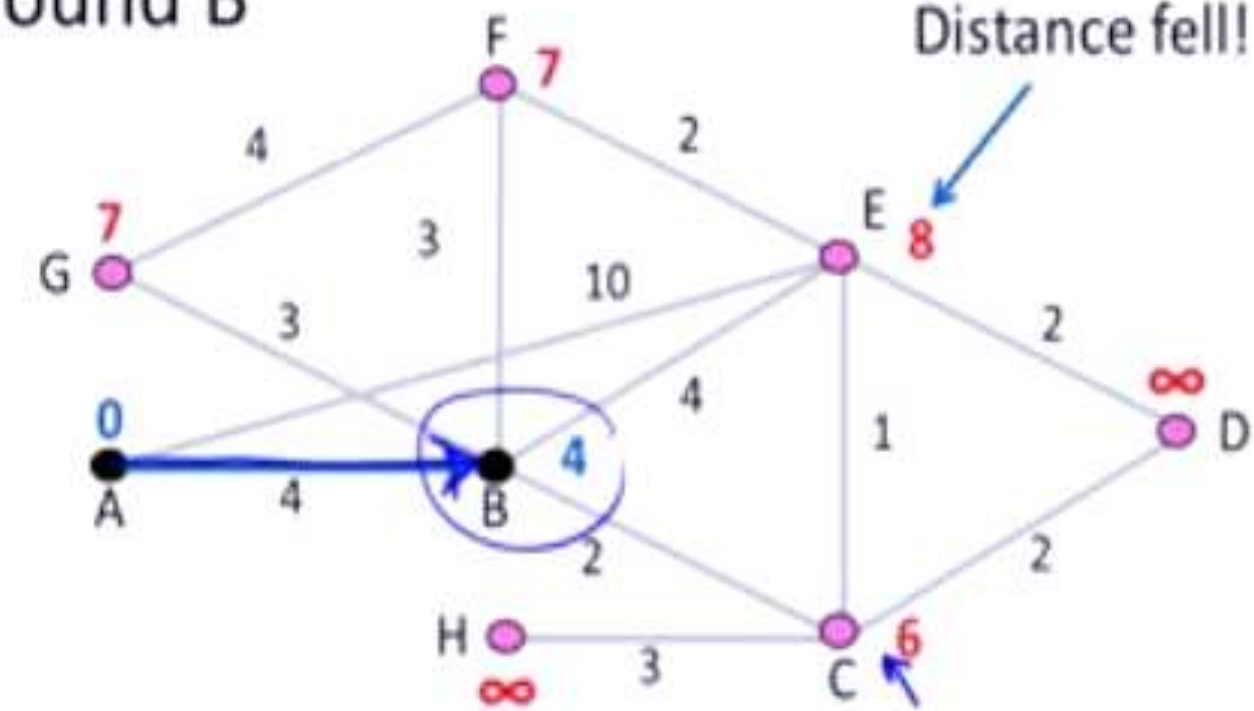
Dijkstra's Algorithm (3)

- Relax around A



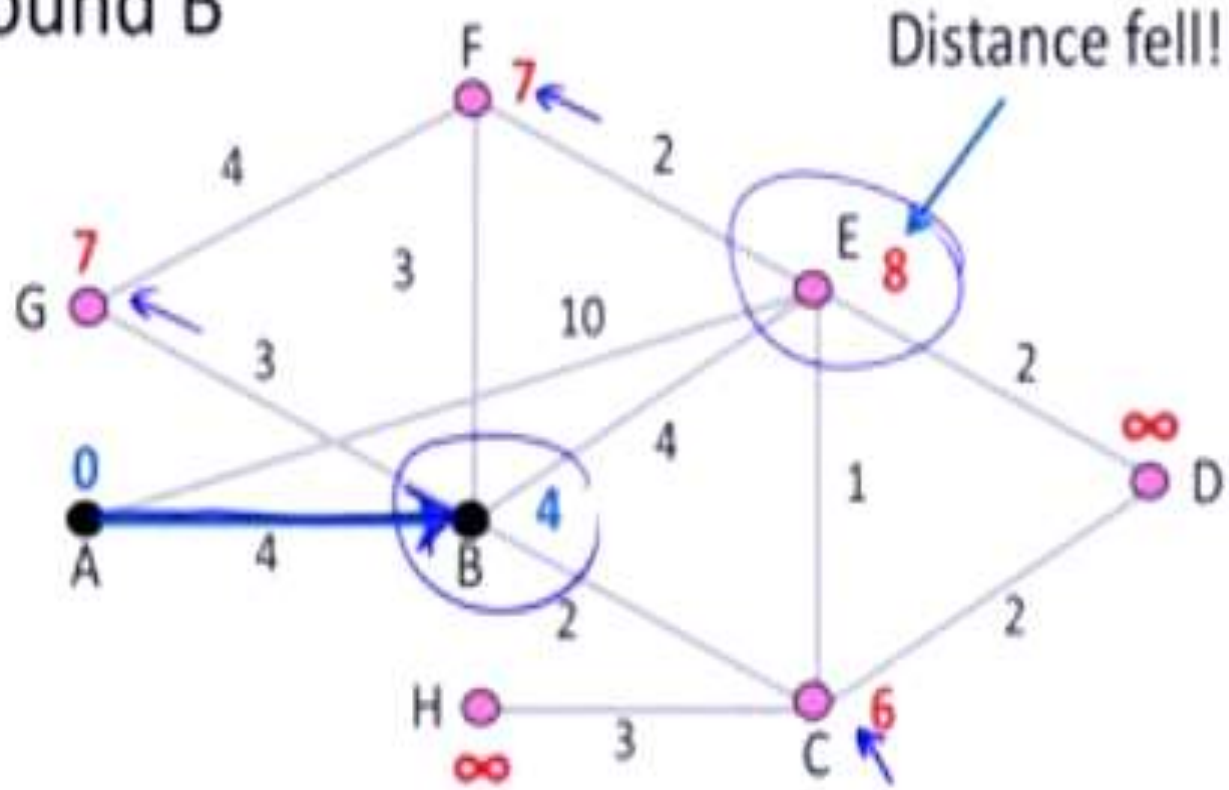
Dijkstra's Algorithm (4)

- Relax around B



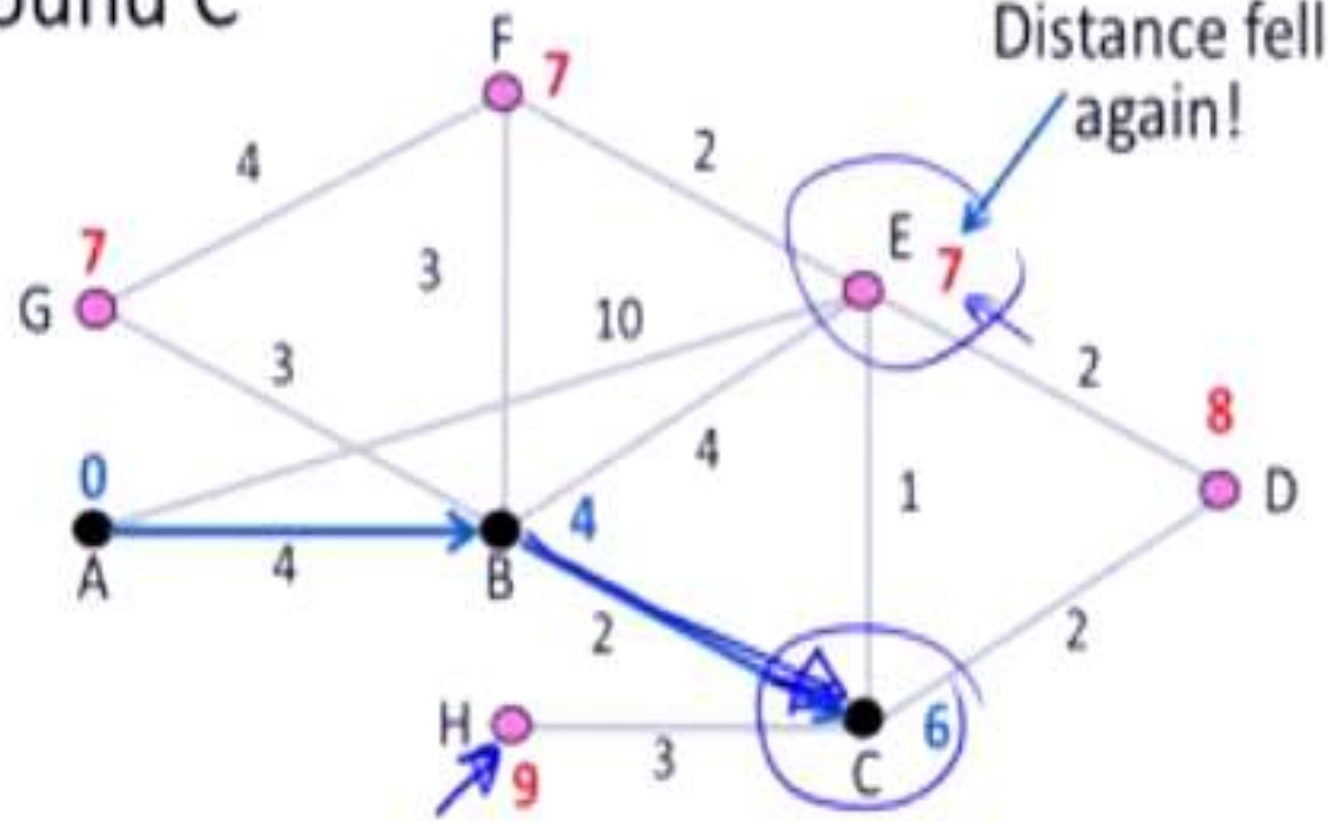
Dijkstra's Algorithm (4)

- Relax around B



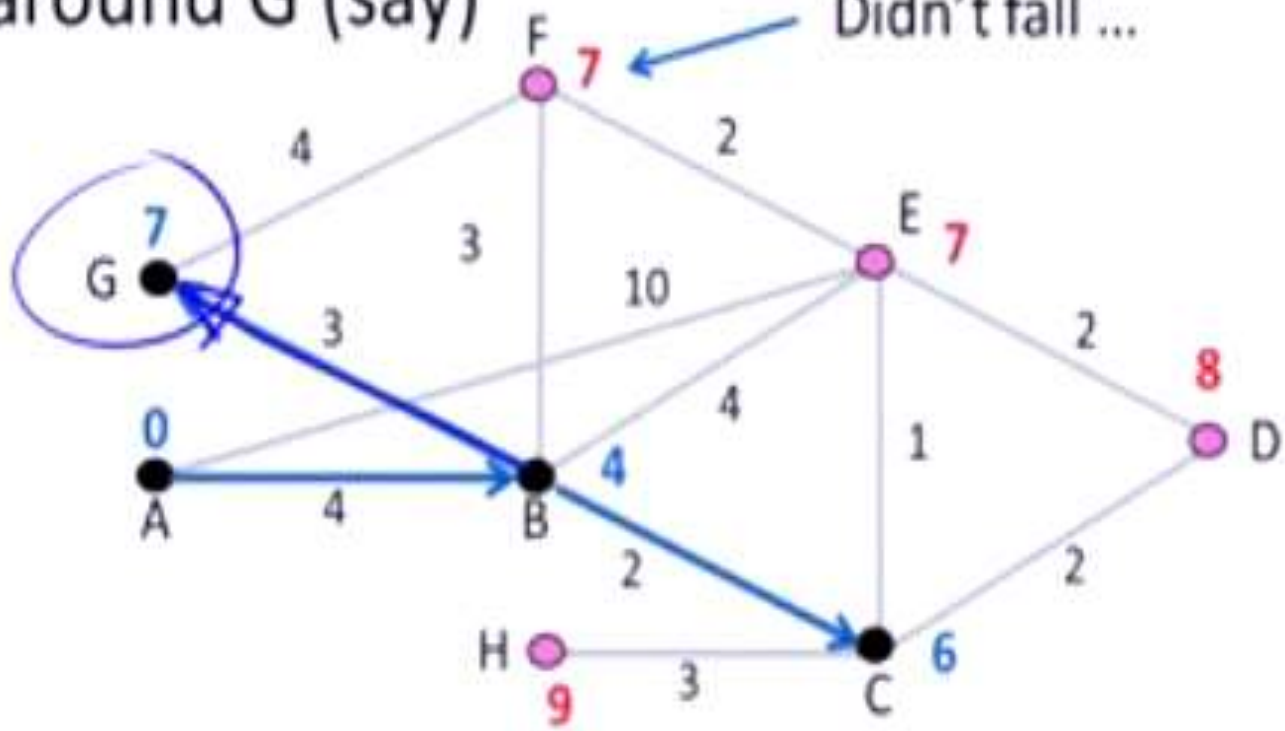
Dijkstra's Algorithm (5)

- Relax around C



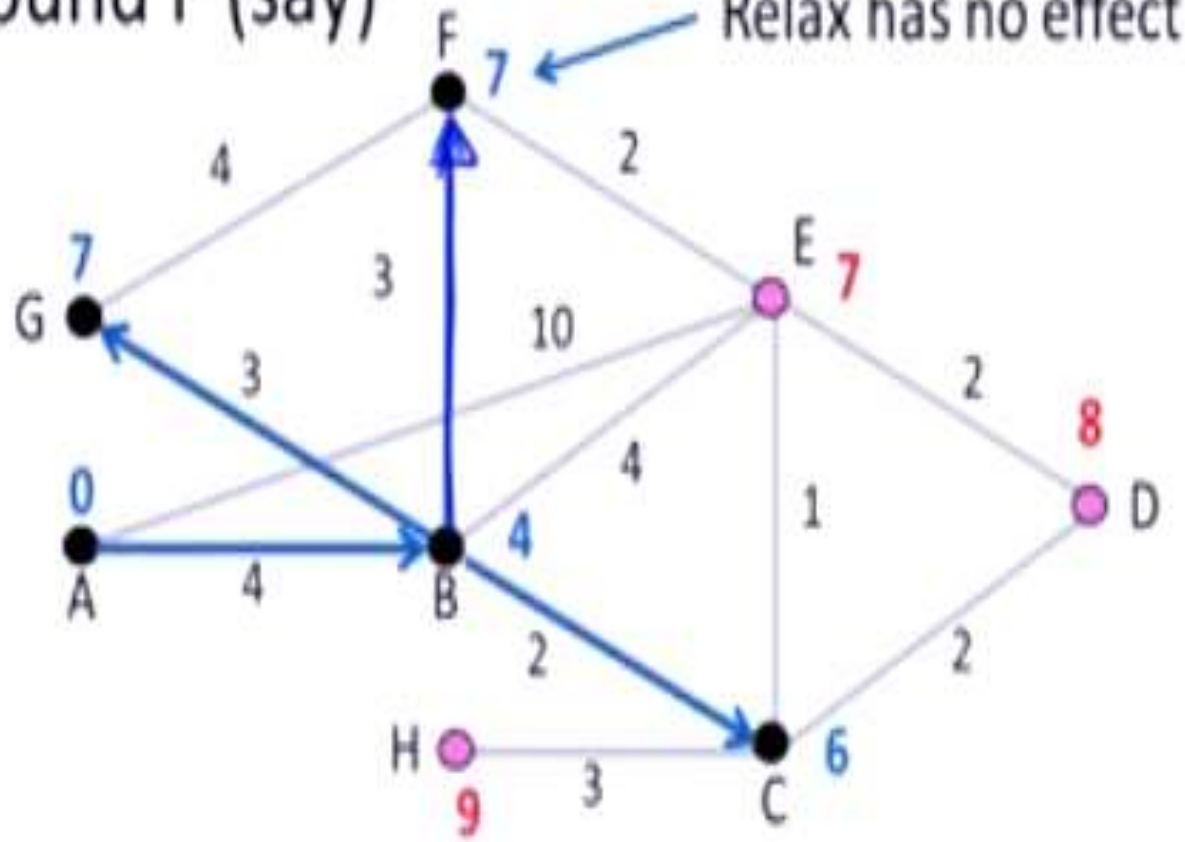
Dijkstra's Algorithm (6)

- Relax around G (say)



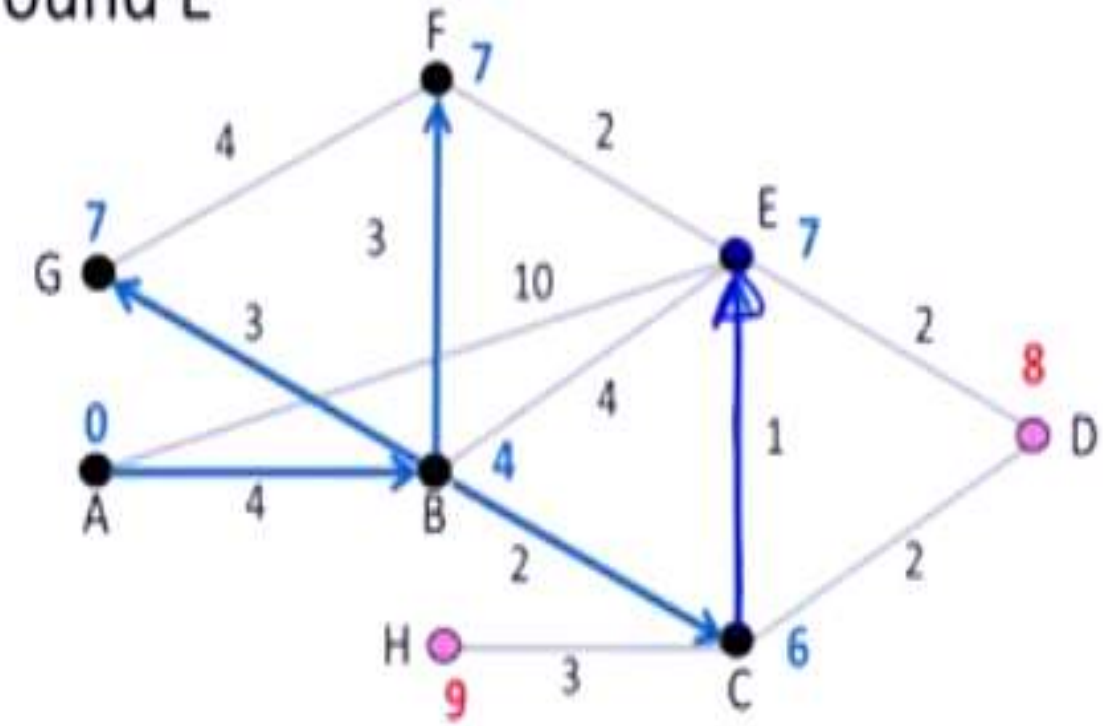
Dijkstra's Algorithm (7)

- Relax around F (say)



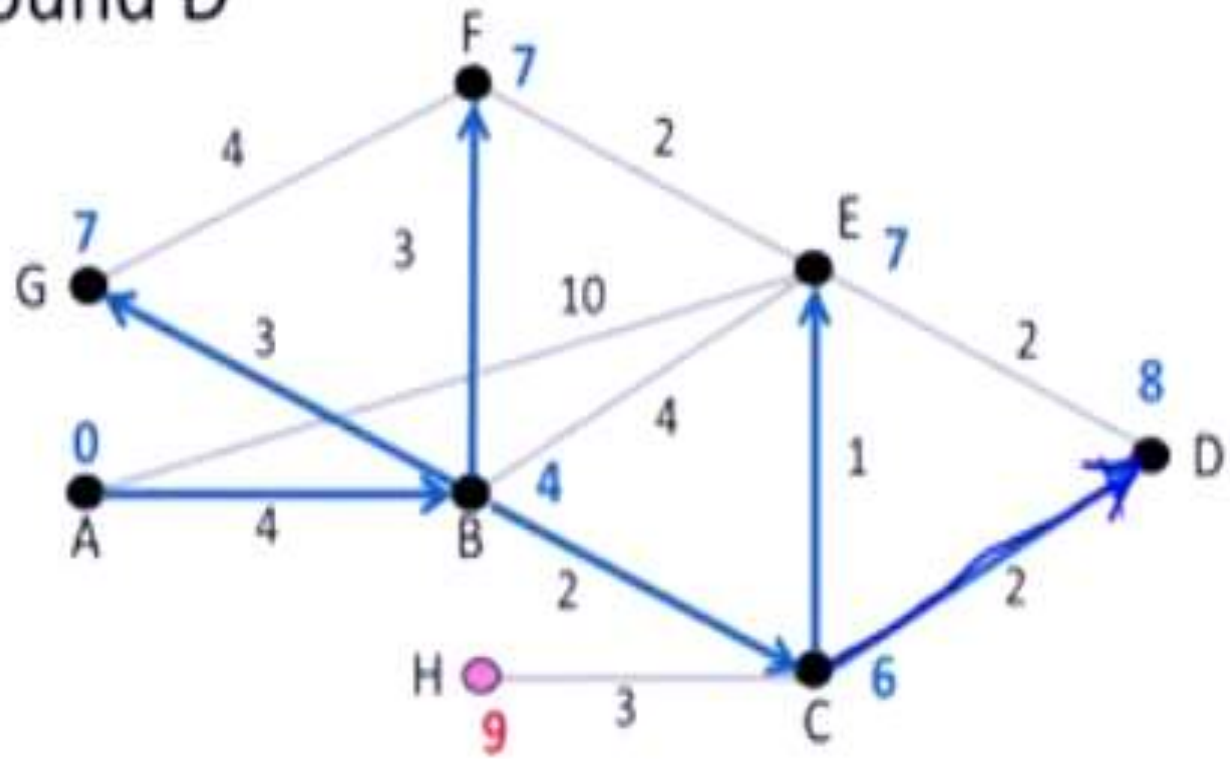
Dijkstra's Algorithm (8)

- Relax around E



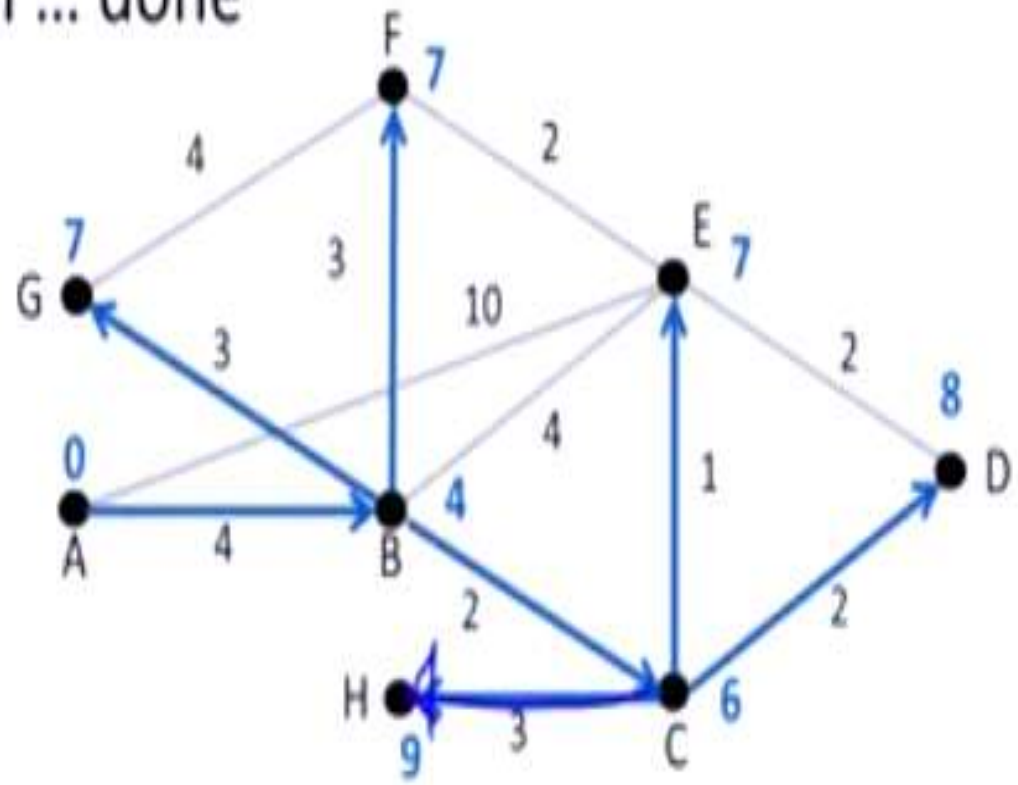
Dijkstra's Algorithm (9)

- Relax around D



Dijkstra's Algorithm (10)

- Finally, H ... done

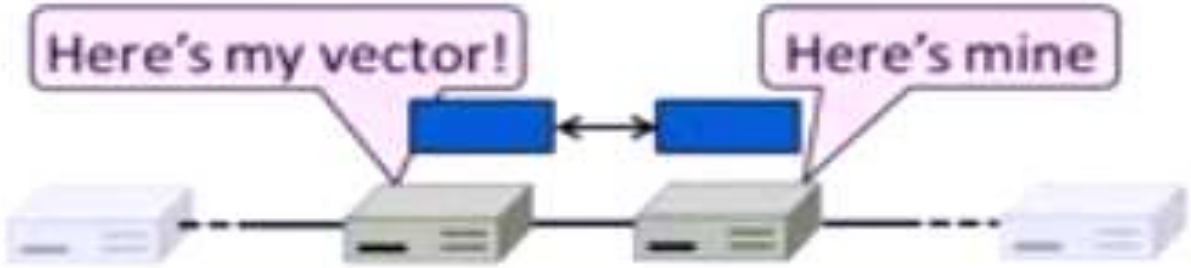


Dijkstra Comments

- Finds shortest paths in order of increasing distance from source
  - Leverages optimality property
- Runtime depends on efficiency of extracting min-cost node
  - Superlinear in network size (grows fast)
- Gives complete source/sink tree
  - More than needed for forwarding!
  - But requires complete topology

Topic

- How to compute shortest paths in a distributed network
  - The Distance Vector (DV) approach



- Simple, early routing approach
  - Used in ARPANET, and RIP
- One of two main approaches to routing
  - Distributed version of Bellman-Ford
  - Works, but very slow convergence after some failures
- Link-state algorithms are now typically used in practice
  - More involved, better behavior

Distance Vector Setting

Each node computes its forwarding table in a distributed setting:

1. Nodes know only the cost to their neighbors; not the topology
2. Nodes can talk only to their neighbors using messages
3. All nodes run the same algorithm concurrently
4. Nodes and links may fail, messages may be lost

Distance Vector Algorithm

Each node maintains a vector of distances (and next hops) to all destinations

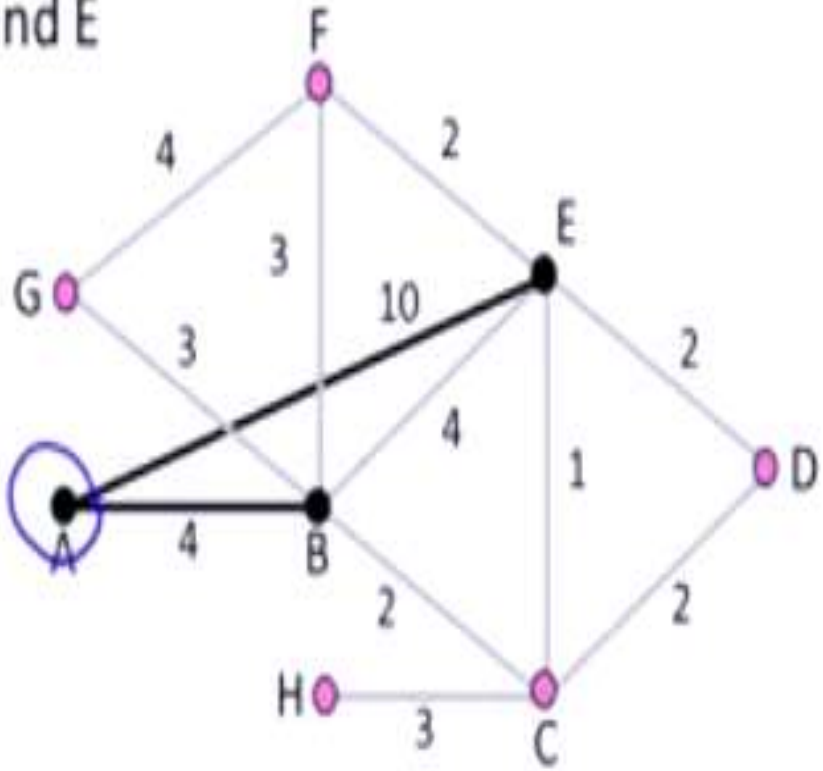
1. Initialize vector with 0 (zero) cost to self, ∞ (infinity) to other destinations
2. Periodically send vector to neighbors
3. Update vector for each destination by selecting the shortest distance heard, after adding cost of neighbor link
   - Use the best neighbor for forwarding

Distance Vector (2)

- Consider from the point of view of node A
  - Can only talk to nodes B and E

Initial vector

To	Cost
A	0
B	∞
C	∞
D	∞
E	∞
F	∞
G	∞
H	∞



Distance Vector (3)

- First exchange with B, E; learn best 1-hop routes

To	B says	E says
A	∞	∞
B	0	∞
C	∞	∞
D	∞	∞
E	∞	0
F	∞	∞
G	∞	∞
H	∞	∞

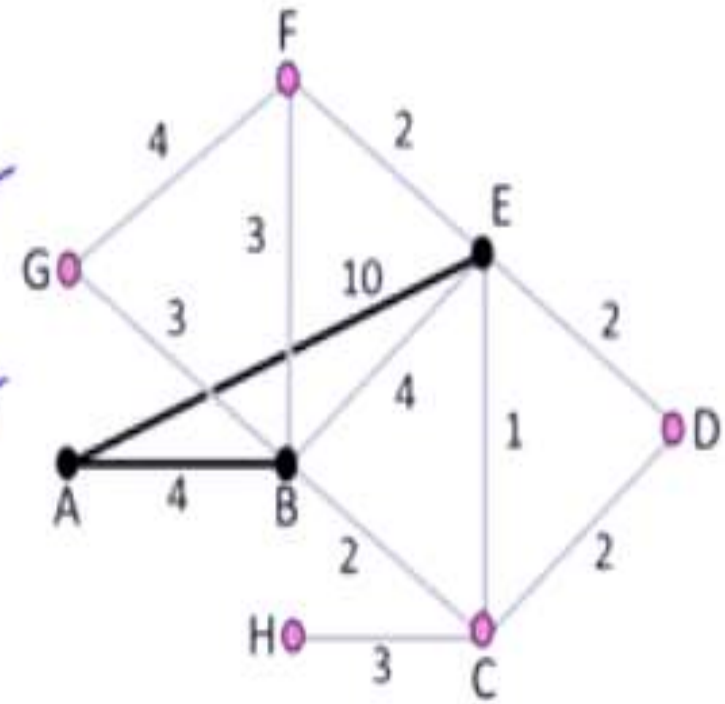
→

B +4	E +10
∞	∞
4	∞
∞	∞
∞	∞
∞	10
∞	∞
∞	∞
∞	∞

→

A's Cost	A's Next
0	--
4	B
∞	--
∞	--
10	E
∞	--
∞	--
∞	--

Learned better route



Distance Vector (4)

- Second exchange; learn best 2-hop routes

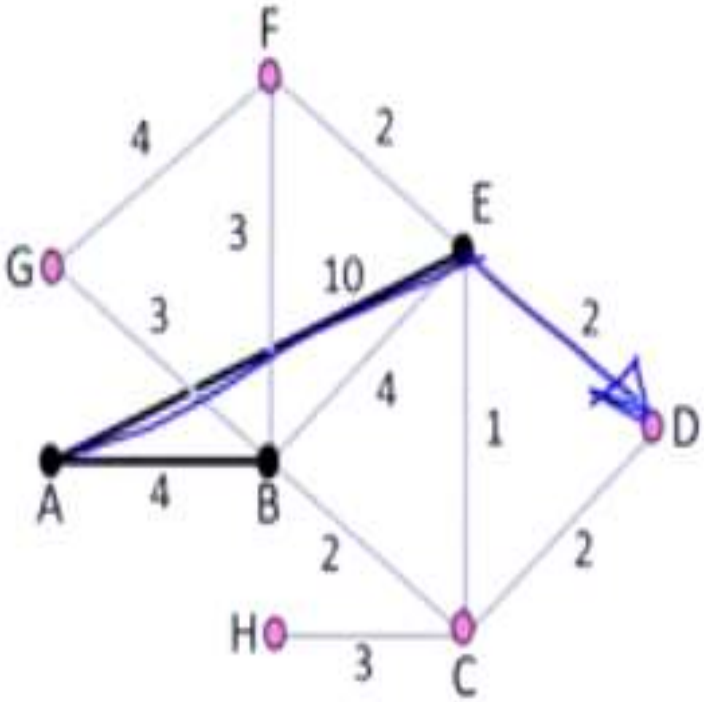
To	B says	E says
A	4	10
B	0	4
C	2	1
D	∞	2
E	4	0
F	3	2
G	3	∞
H	∞	∞

→

B +4	E +10
8	20
4	14
6	11
∞	12
8	10
7	12
7	∞
∞	∞

→

A's Cost	A's Next
0	--
4	B
6	B
12	E
8	B
7	B
7	B
∞	--



Distance Vector (4)

- Second exchange; learn best 2-hop routes

To	B says	E says
A	4	10
B	0	4
C	2	1
D	∞	2
E	4	0
F	3	2
G	3	∞
H	∞	∞

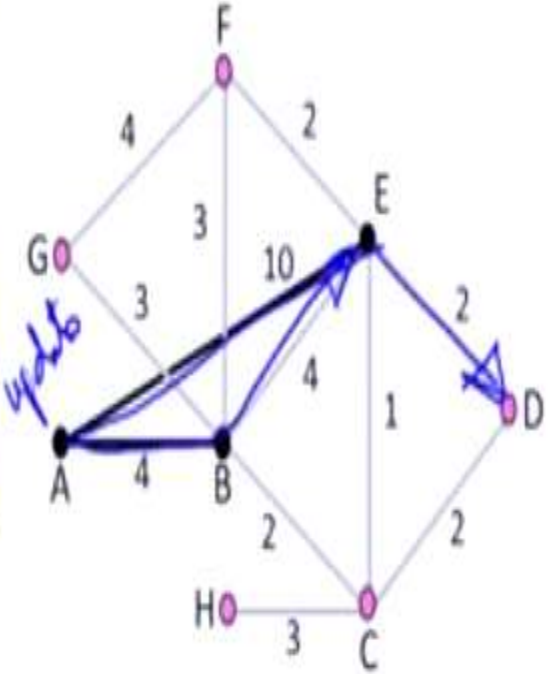
→

B +4	E +10
8	20
4	14
6	11
∞	12
8	10
7	12
7	∞
∞	∞

→

A's Cost	A's Next
0	--
4	B
6	B
12	E
8	B
7	B
7	B
∞	--

update



Distance Vector (4)

- Third exchange; learn best 3-hop routes

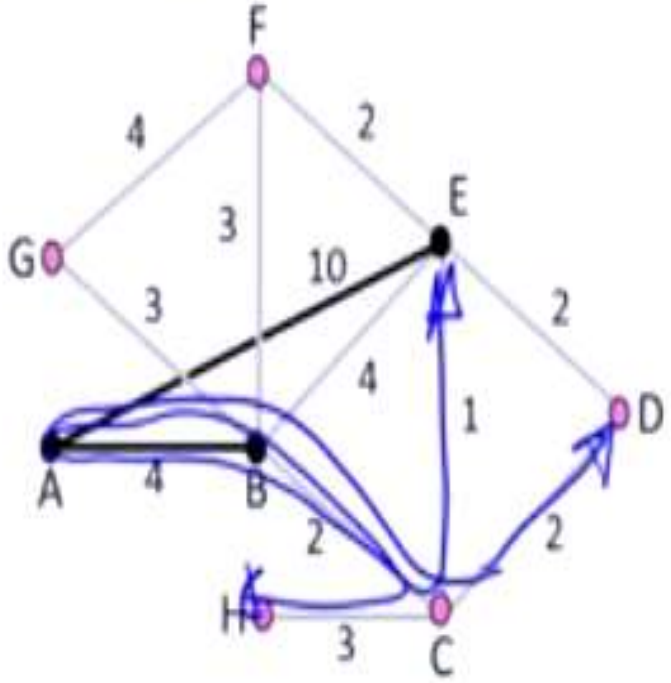
To	B says	E says
A	4	8
B	0	3
C	2	1
D	4	2
E	3	0
F	3	2
G	3	6
H	5	4

→

B +4	E +10
8	18
4	13
6	11
8	12
7	10
7	12
7	16
9	14

→

A's Cost	A's Next
0	--
4	B
6	B
8	B
7	B
7	B
7	B
9	B



Distance Vector (5)

- Subsequent exchanges; converged

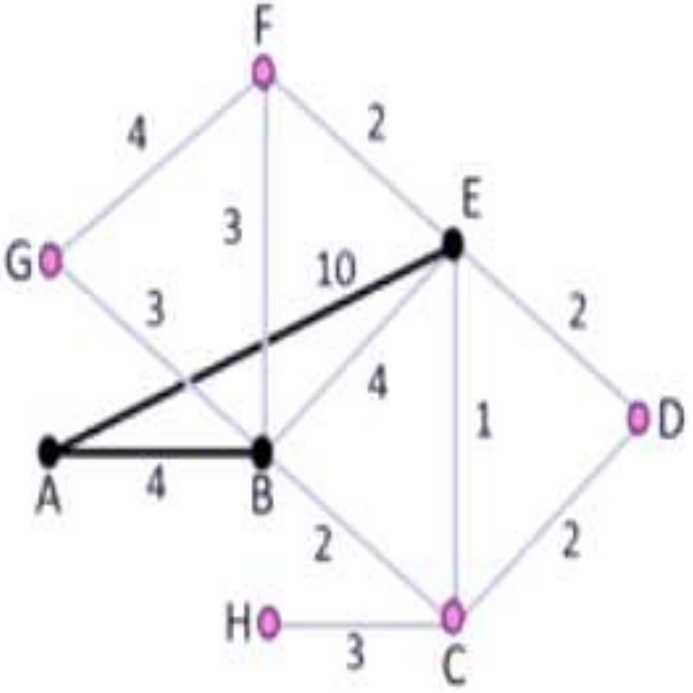
To	B says	E says
A	4	7
B	0	3
C	2	1
D	4	2
E	3	0
F	3	2
G	3	6
H	5	4

→

B +4	E +10
8	17
4	13
6	11
8	12
7	10
7	12
7	16
9	14

→

A's Cost	A's Next
0	--
4	B
6	B
8	B
8	B
7	B
7	B
9	B



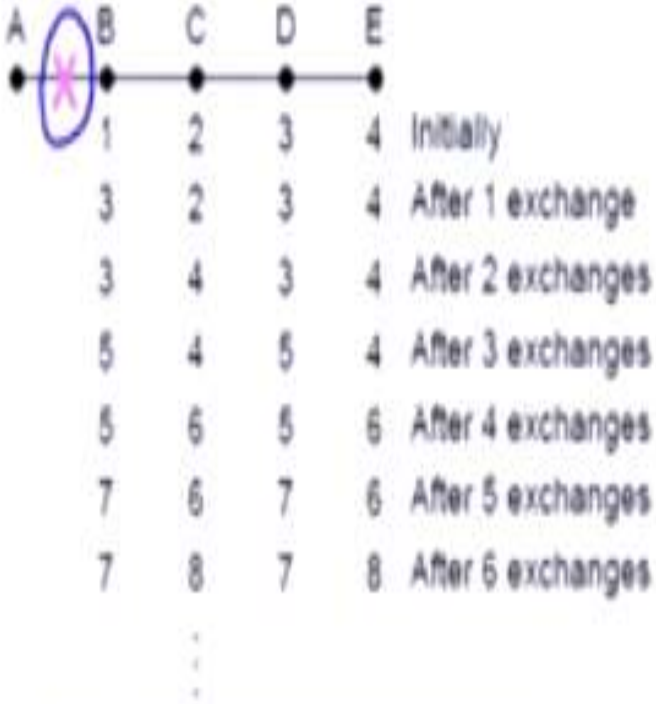
Distance Vector Dynamics

- Adding routes:
  - News travels one hop per exchange
- Removing routes
  - When a node fails, no more exchanges, other nodes forget
- But partitions (unreachable nodes in divided network) are a problem
  - "Count to infinity" scenario

DV Dynamics (2)

- Good news travels quickly, bad news slowly (inferred)

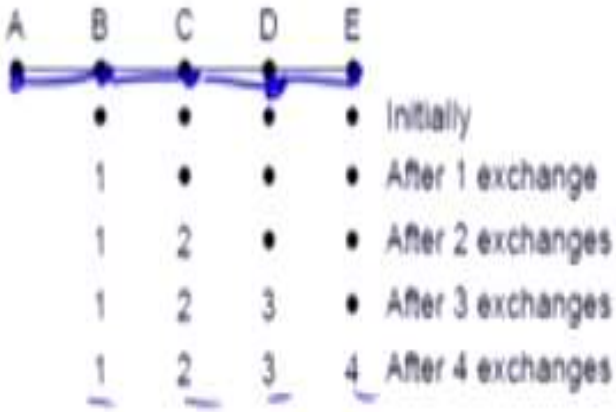
Desired convergence



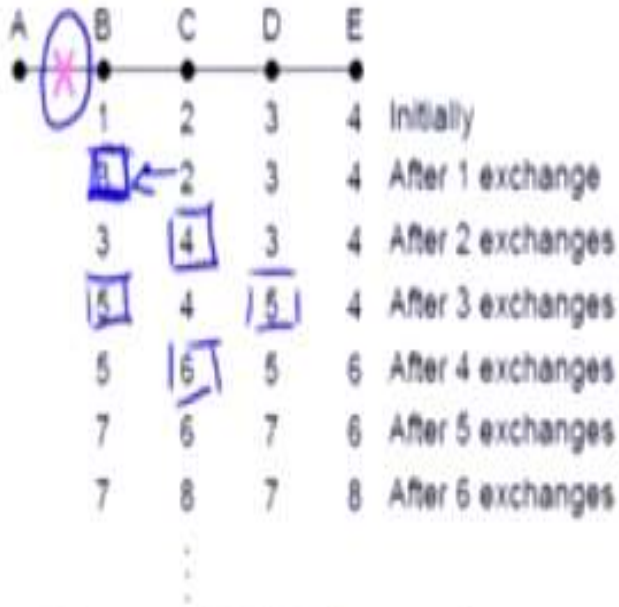
"Count to infinity" scenario

DV Dynamics (2)

- Good news travels quickly, bad news slowly (inferred)



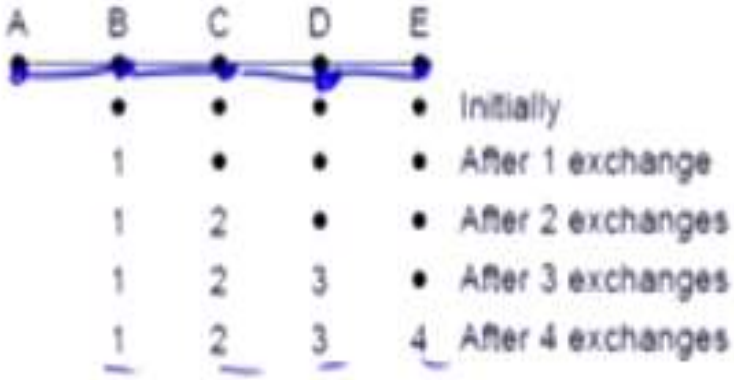
Desired convergence



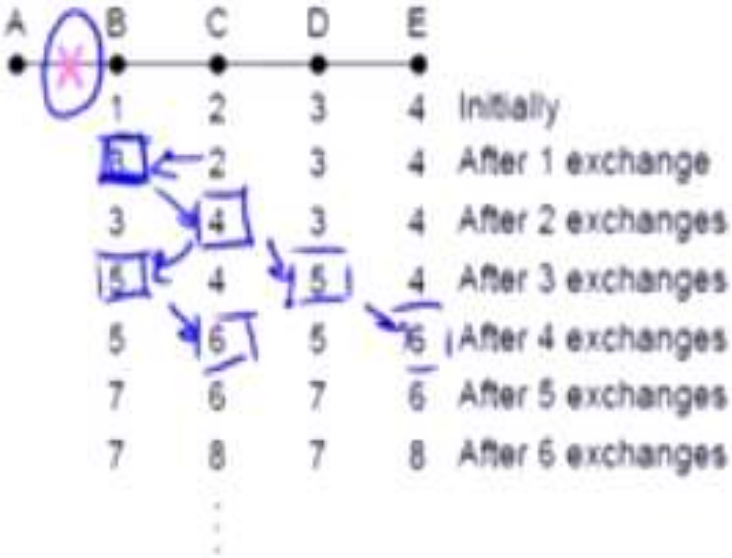
"Count to infinity" scenario

DV Dynamics (2)

- Good news travels quickly, bad news slowly (inferred)

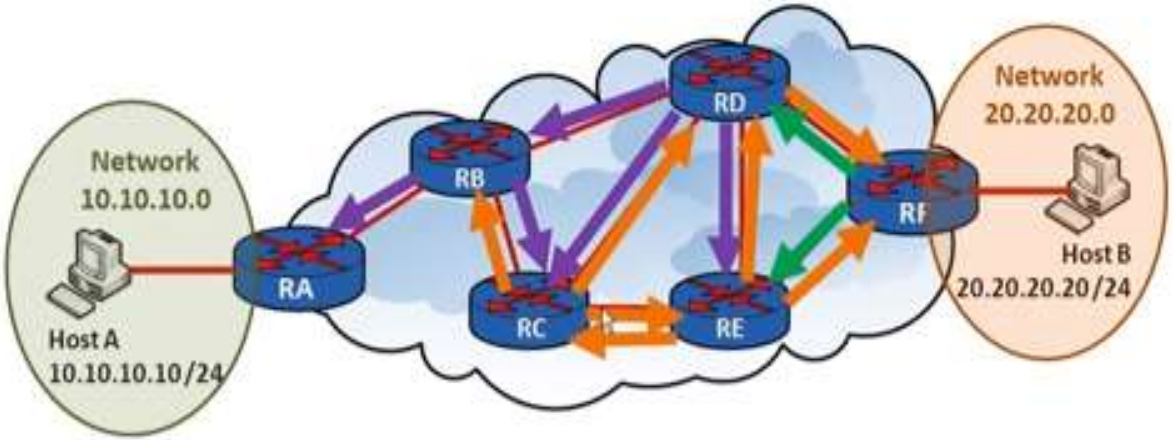


Desired convergence



"Count to infinity" scenario

Routing Operations



DV Dynamics (3)

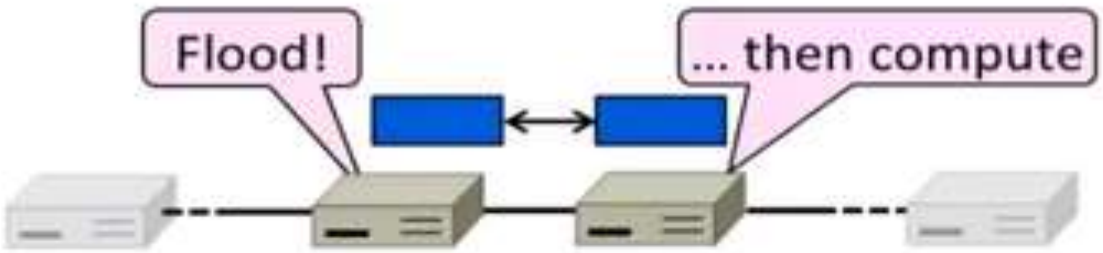
- Various heuristics to address
  - e.g., "Split horizon, poison reverse" (Don't send route back to where you learned it from.)
- But none are very effective
  - Link state now favored in practice
  - Except when very resource-limited

RIP (Routing Information Protoco

- DV protocol with hop count as metric
  - Infinity is 16 hops; limits network size
  - Includes split horizon, poison reverse
- Routers send vectors every 30 seconds
  - Runs on top of UDP
  - Time-out in 180 secs to detect failures
- RIPv1 specified in RFC1058 (1988)

Topic

- How to compute shortest paths in a distributed network
  - The Link-State (LS) approach



Link-State Routing

- One of two approaches to routing
  - Trades more computation than distance vector for better dynamics
- Widely used in practice
  - Used in Internet/ARPANET from 1979
  - Modern networks use OSPF and IS-IS

Link-State Setting

Nodes compute their forwarding table in the same distributed setting as for distance vector:

1. Nodes know only the cost to their neighbors; not the topology
2. Nodes can talk only to their neighbors using messages
3. All nodes run the same algorithm concurrently
4. Nodes/links may fail, messages may be lost

Link-State Algorithm

Proceeds in two phases:

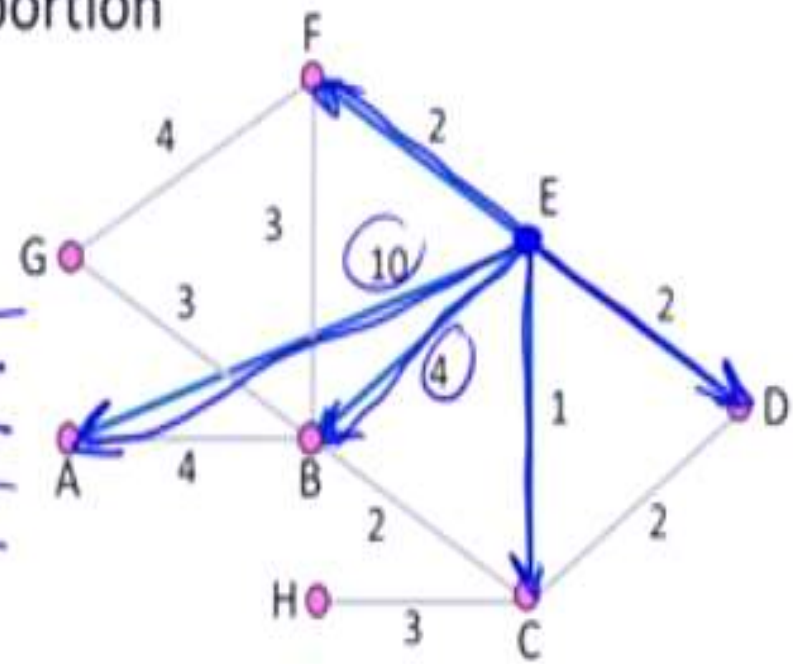
1. Nodes flood topology in the form of link state packets
   - Each node learns full topology
2. Each node computes its own forwarding table
   - By running Dijkstra (or equivalent)

Phase 1: Topology Dissemination

- Each node floods link state packet (LSP) that describes their portion of the topology

Node E's LSP flooded to A, B, C, D, and F

	Seq. #
A	10
B	4
C	1
D	2
F	2

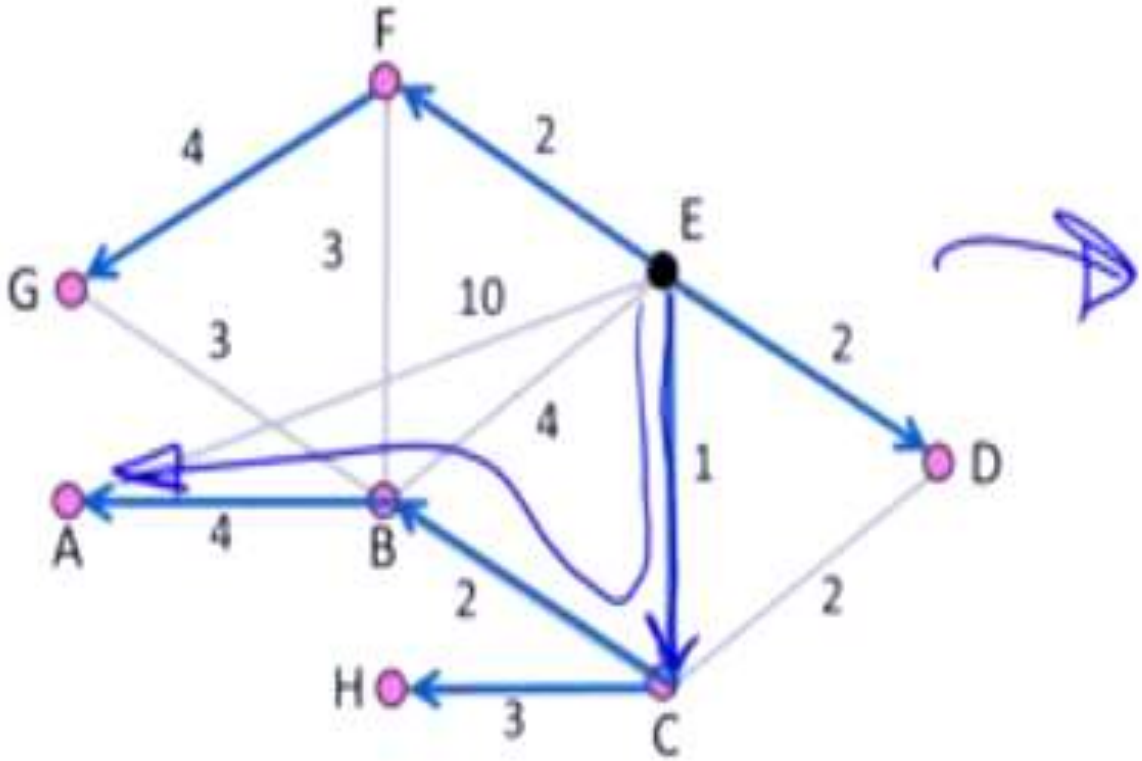


Phase 2: Route Computation

- Each node has full topology
  - By combining all LSPs
- Each node simply runs Dijkstra
  - Some replicated computation, but finds required routes directly
  - Compile forwarding table from sink/source tree
  - That's it folks!

Forwarding Table

Source Tree for E (from Dijkstra)



E's Forwarding Table

To	Next
A	C
B	C
C	C
D	D
E	--
F	F
G	F
H	C

Handling Changes

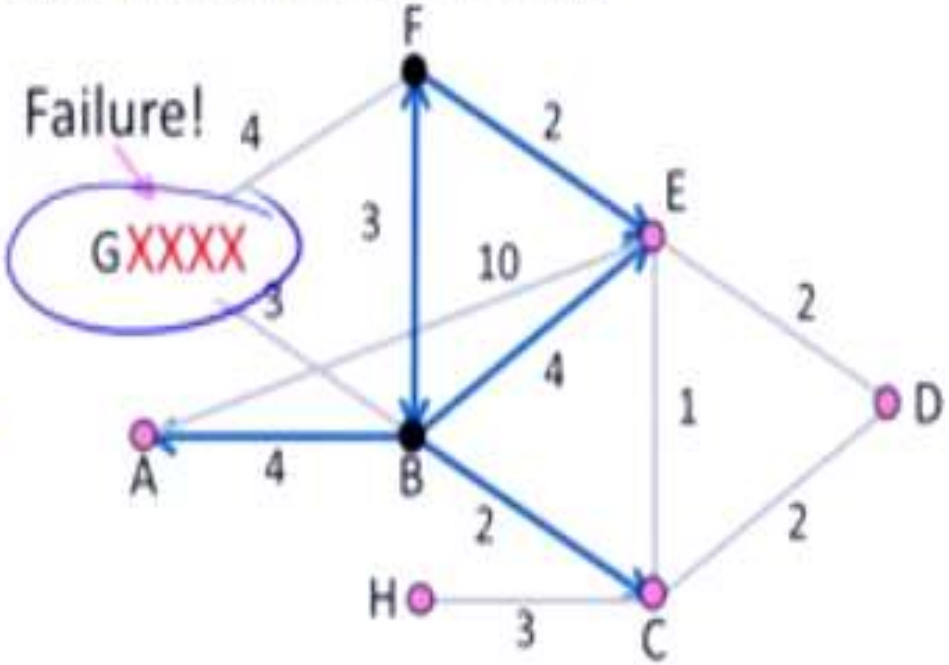
- On change, flood updated LSPs, and re-compute routes
  - E.g., nodes adjacent to failed link or node initiate

B's LSP

Seq. #	
A	4
C	2
E	4
F	3
G	∞

F's LSP

Seq. #	
B	3
E	2
G	∞



Handling Changes (2)

- Link failure
  - Both nodes notice, send updated LSPs
  - Link is removed from topology
- Node failure
  - All neighbors notice a link has failed
  - Failed node can't update its own LSP
  - But it is OK: all links to node removed

Handling Changes (3)

- Addition of a link or node
  - Add LSP of new node to topology
  - Old LSPs are updated with new link
- Additions are the easy case ...

Link-State Complications

- Things that can go wrong:
  - Seq. number reaches max, or is corrupted
  - Node crashes and loses seq. number
  - Network partitions then heals
- Strategy:
  - Include age on LSPs and forget old information that is not refreshed
- Much of the complexity is due to handling corner cases (as usual!)

DV/LS Comparison

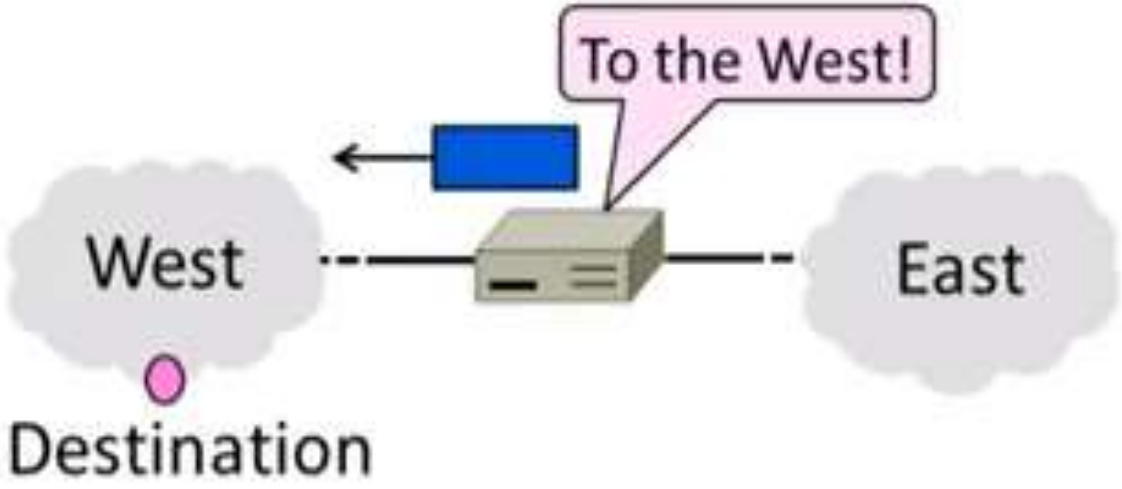
Goal	Distance Vector	Link-State
Correctness	Distributed Bellman-Ford	Replicated Dijkstra
Efficient paths	Approx. with shortest paths	Approx. with shortest paths
Fair paths	Approx. with shortest paths	Approx. with shortest paths
Fast convergence	Slow – many exchanges	Fast – flood and compute
Scalability	Excellent – storage/compute	Moderate – storage/compute

IS-IS and OSPF Protocols

- Widely used in large enterprise and ISP networks
  - IS-IS = Intermediate System to Intermediate System
  - OSPF = Open Shortest Path First
- Link-state protocol with many added features
  - E.g., "Areas" for scalability

Topic

- How to scale routing with hierarchy in the form of regions
  - Route to regions, not individual nodes



Internet Routing Growth

- Internet growth translates into routing table growth (even using prefixes) …

Source: By Mro (Own work), CC-BY-SA-3.0, via Wikimedia Commons

Impact of Routing Growth

1. Forwarding tables grow
   - Larger router memories, may increase lookup time
2. Routing messages grow
   - Need to keeps all nodes informed of larger topology
3. Routing computation grows
   - Shortest path calculations grow faster than the size of the network

Techniques to Scale Routing

1. IP prefixes
   – Route to blocks of hosts

   Last time

2. Network hierarchy
   – Route to network regions

   This time

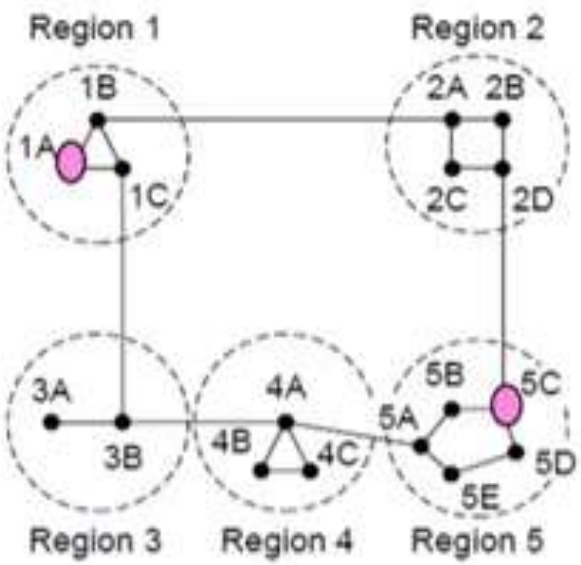
3. IP prefix aggregation
   – Combine, and split, prefixes

   Next time

Hierarchical Routing

- Introduce a larger routing unit
  - IP prefix (hosts) ← from one host
  - Region, e.g., ISP network
- Route first to the region, then to the IP prefix within the region
  - Hide details within a region from outside of the region

Hierarchical Routing (2)



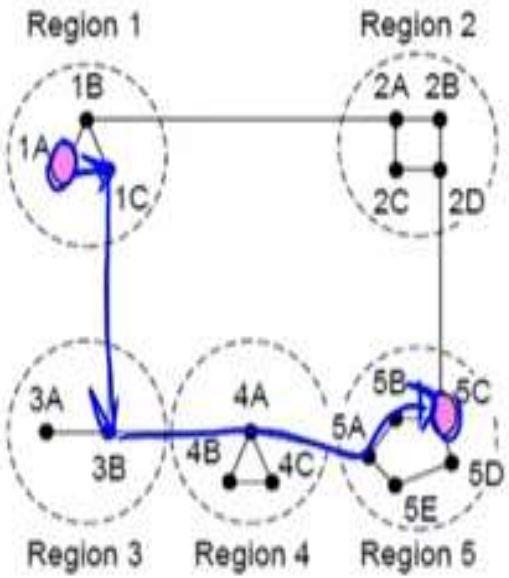
Full table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

Hierarchical table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

Hierarchical Routing (2)



Full table for 1A

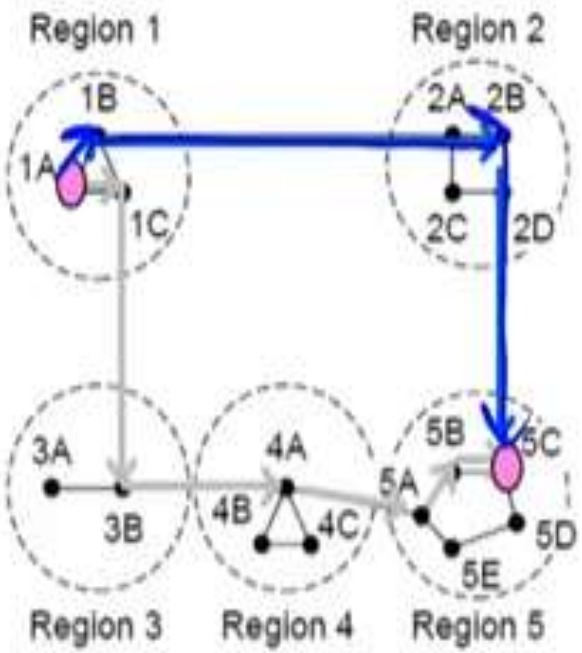
Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

Hierarchical table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
	1C	4

Hierarchical Routing (4)

- Penalty is longer paths



Full table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

Hierarchical table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

1C is best route to region 5, except for destination 5C

Distance Vector Routing Protocol	Link State Routing Protocol
Entire routing table is sent as an update	Updates are incremental & entire routing table is not sent as update
Send periodic update (Every fixed interval of time) at every 30 or 90 second	Updates are triggered not periodic
Update are broadcasted (not in RIP v2)	Updates are multicasted

Free Software
Download Free PC Manager Software. Easy File Transfer. Download Now!
mobogenie.com/download-software
Ads by Goo

Distance Vector Vs Link State

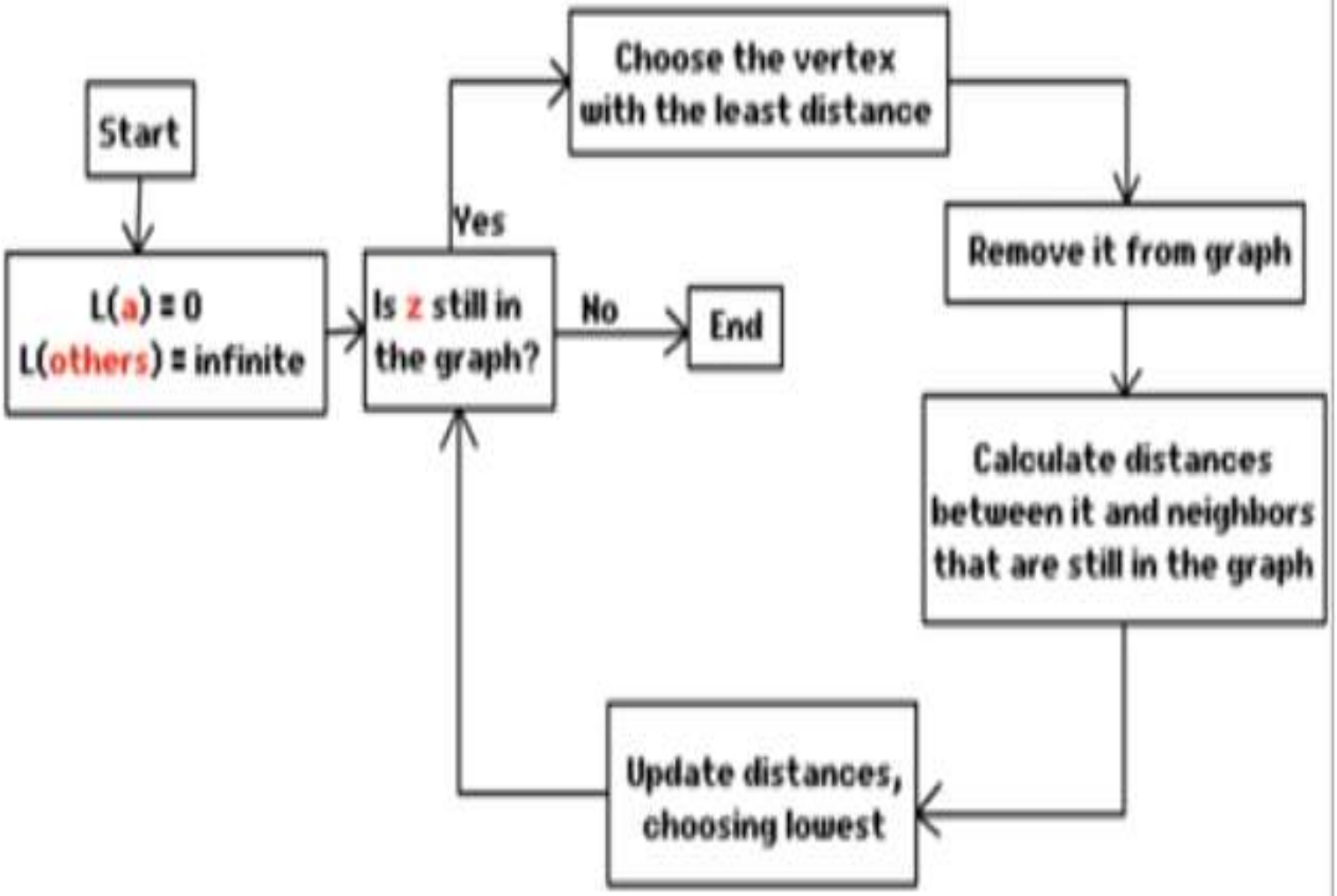
Updates are sent to directly connected neighbour only	Update are sent to entire network
Routers don't have end to end visibility of entire network	Routers have visibility of entire network of that area only.
Prone to routing loops	No routing loops

Dijkstra's algorithm calculates the least distance from a starting vertex a to a destination vertex z from a weighted graph.

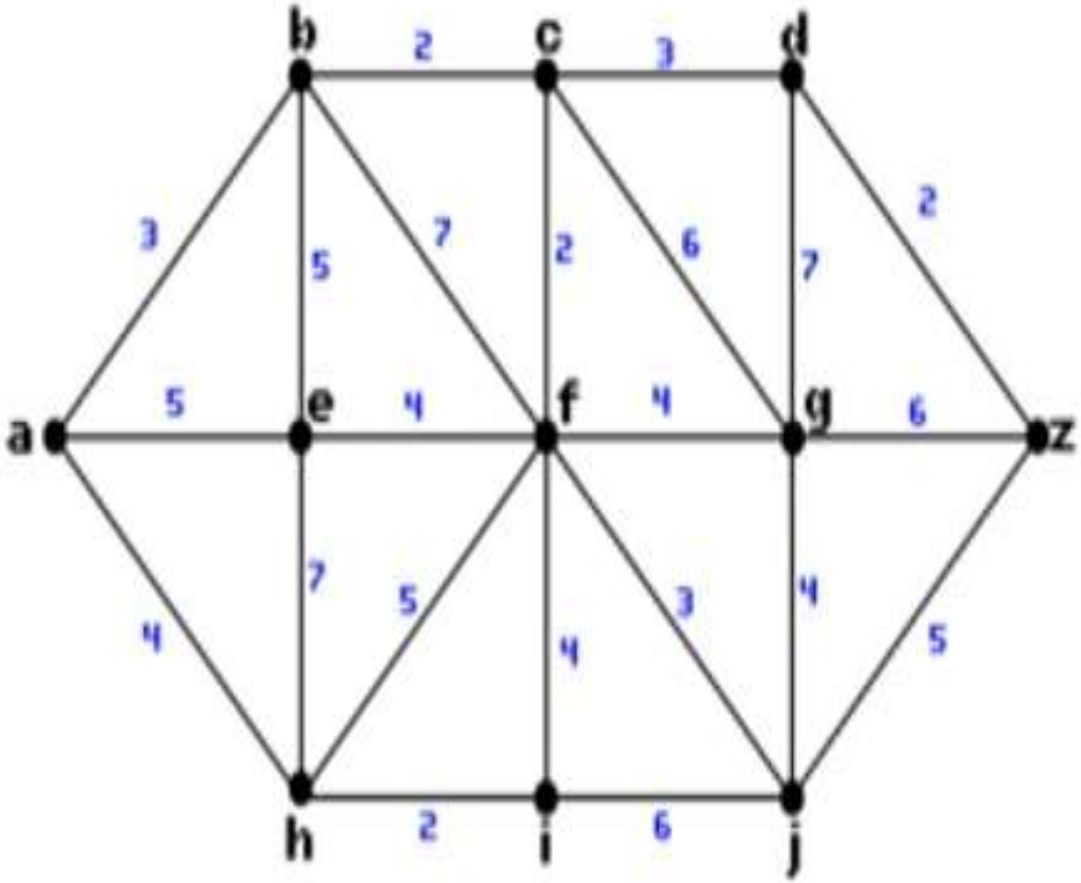
Input: A weighted graph, starting and ending vertices and the least distances L for each vertex (variables that are changed in the execution of the algorithm).

Output: L(z), the least distance from a to z.

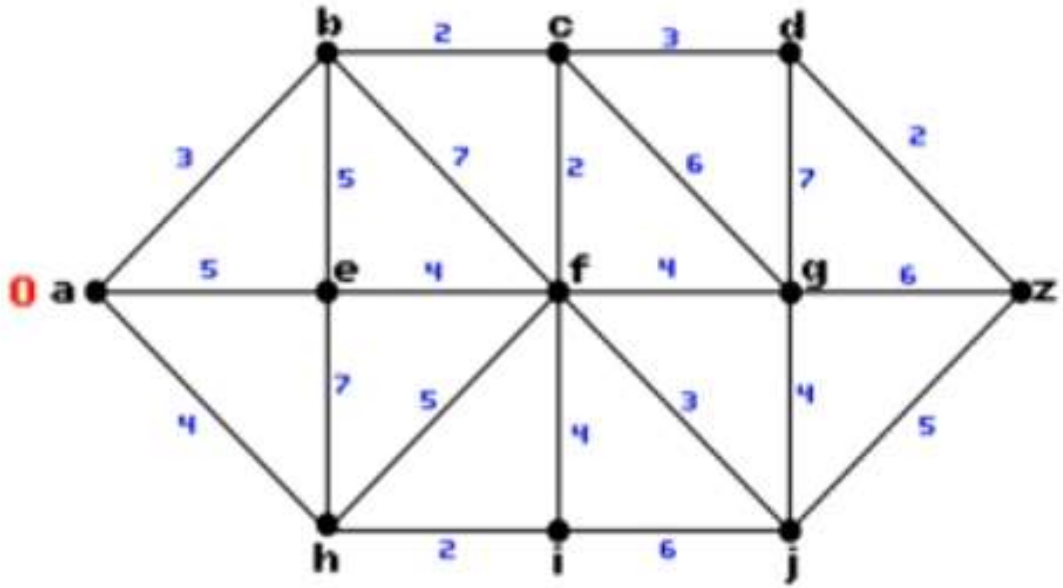
Dijkstra's algorithm goes like this:



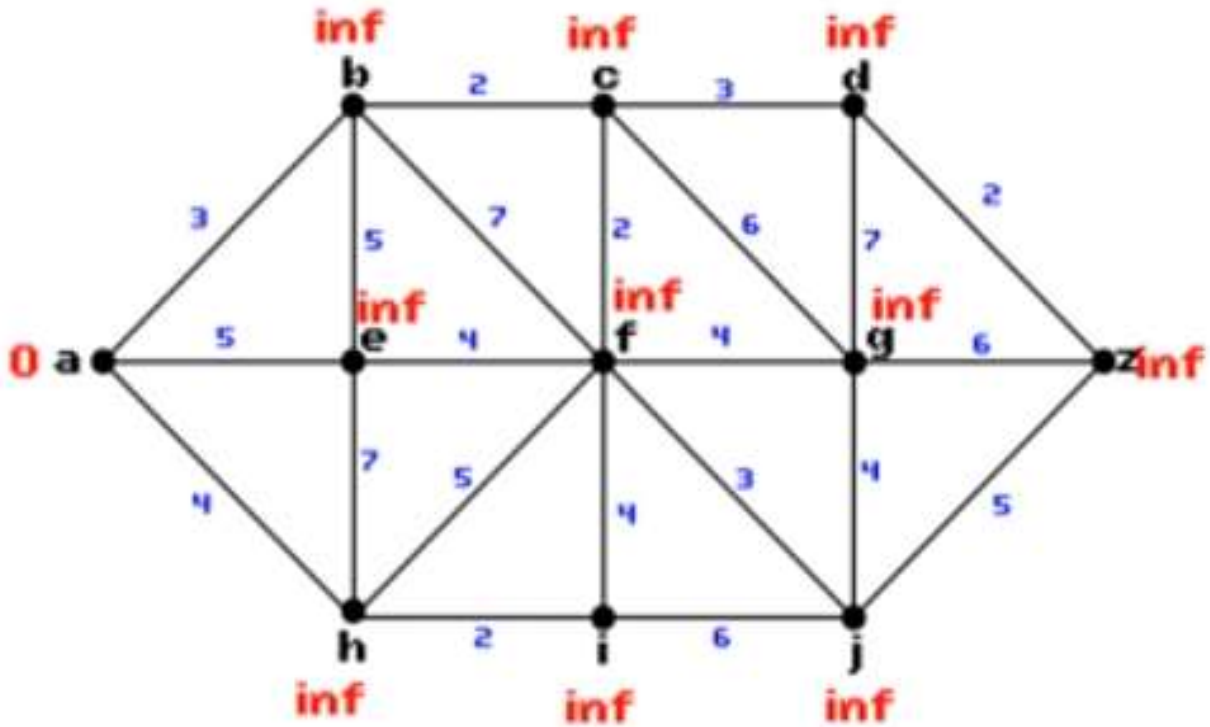
Example: find the shortest path from a to z.



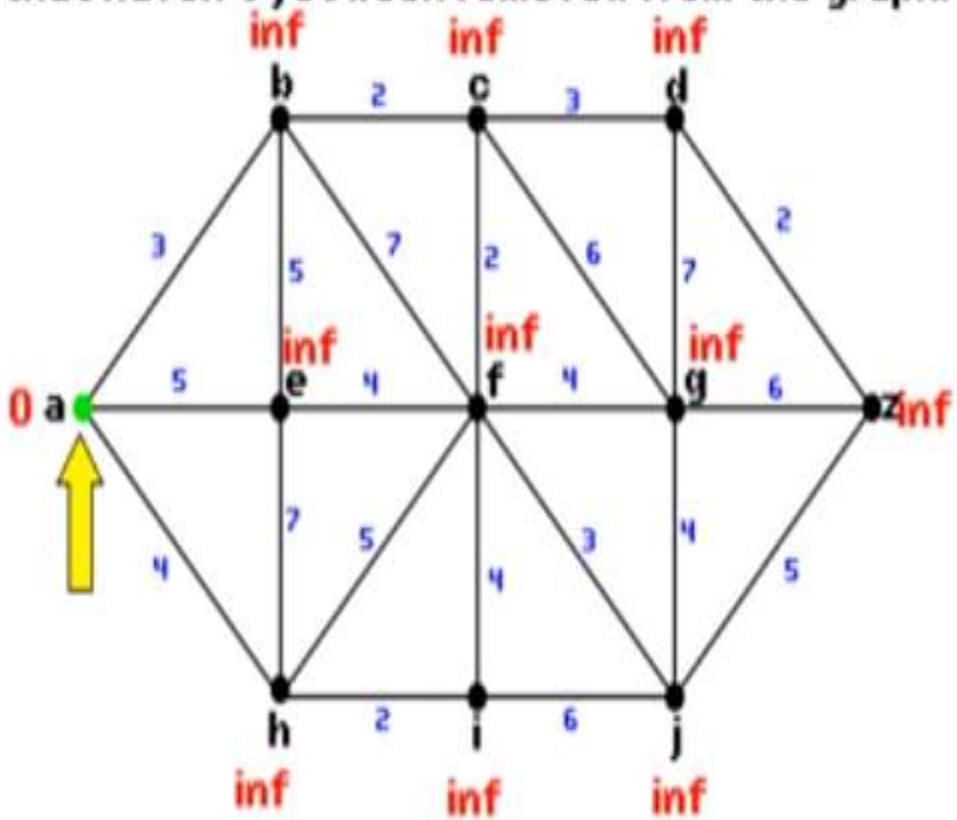
Set distance infinite for the rest.



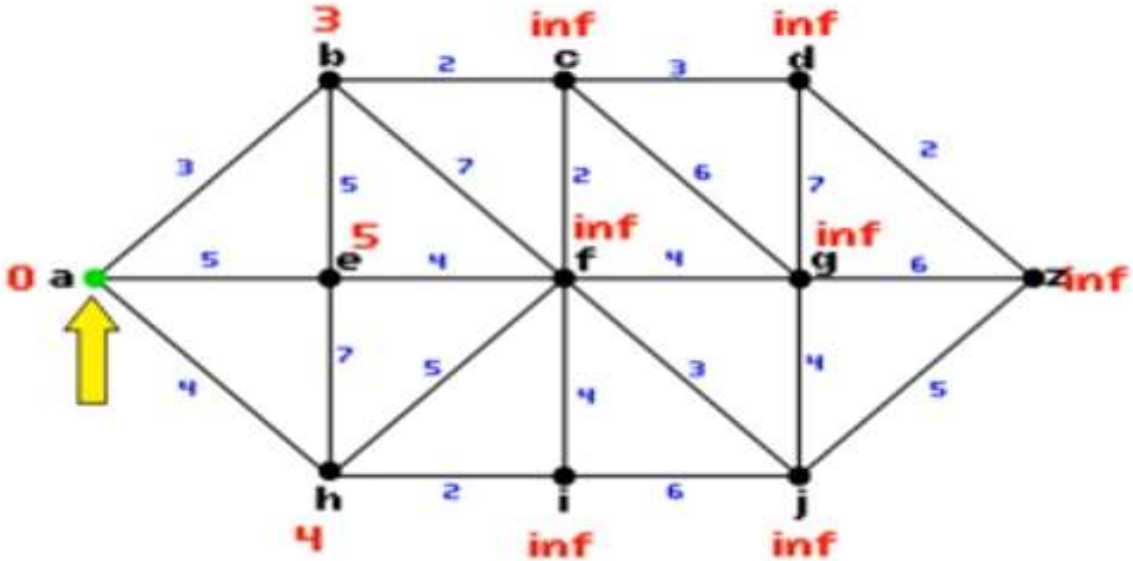
Pick the vertex with the least distance value.



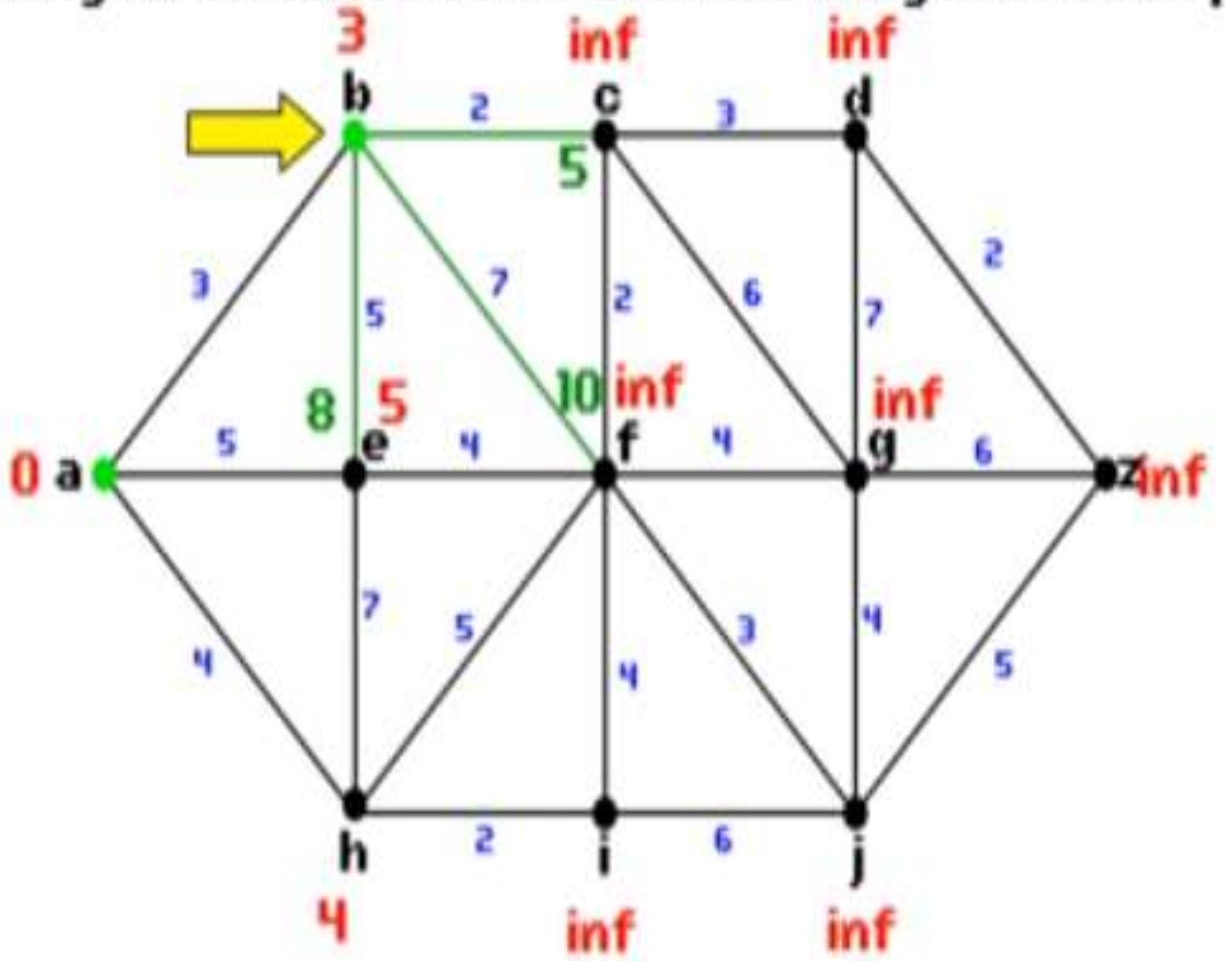
Calculate distance from chosen vertex to adjacent ones that haven't yet been removed from the graph.



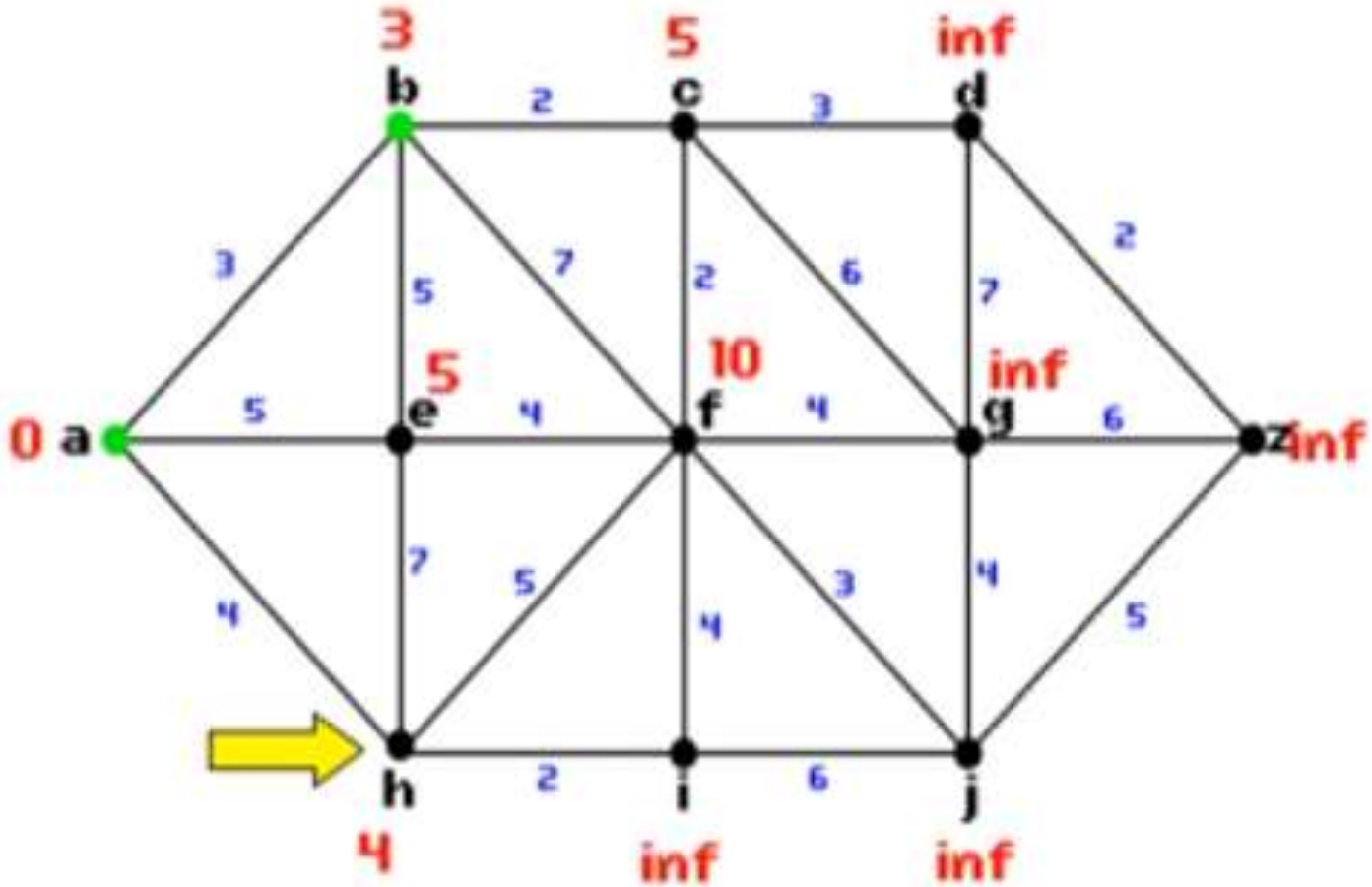
Update the distance of those vertices. Choose lowest value.



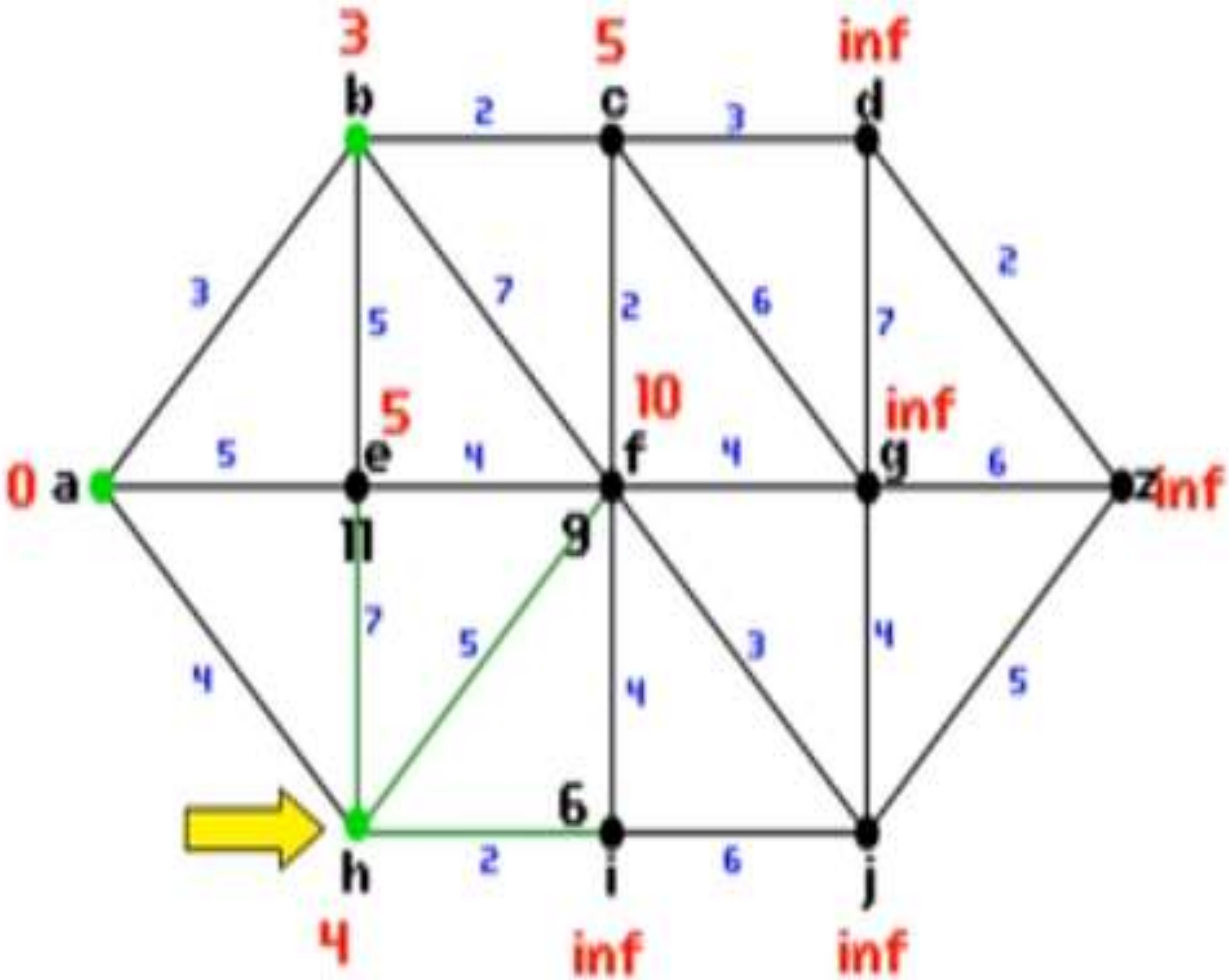
The distance of the adjacent vertices is calculated by adding its distance value with the weigth of each path.



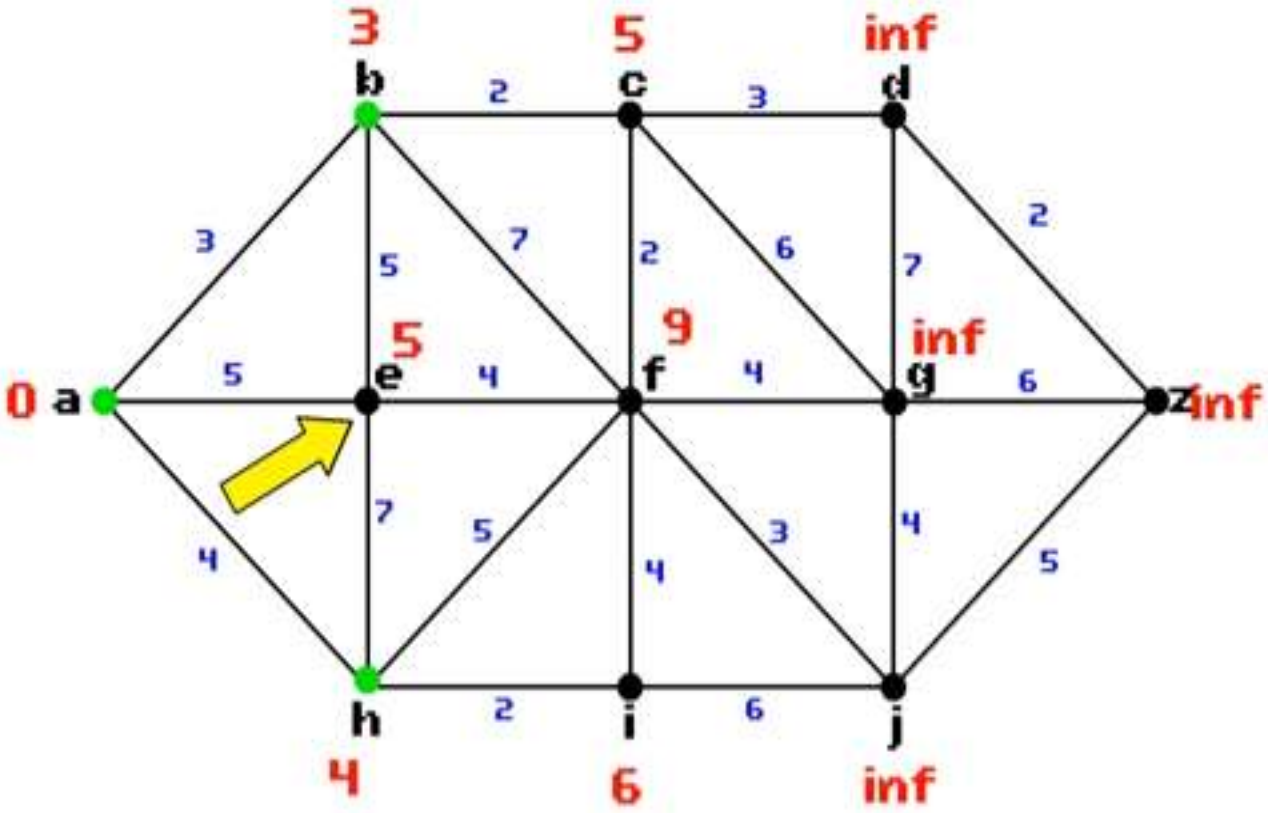
Repeat! Choose vertex with least distance.



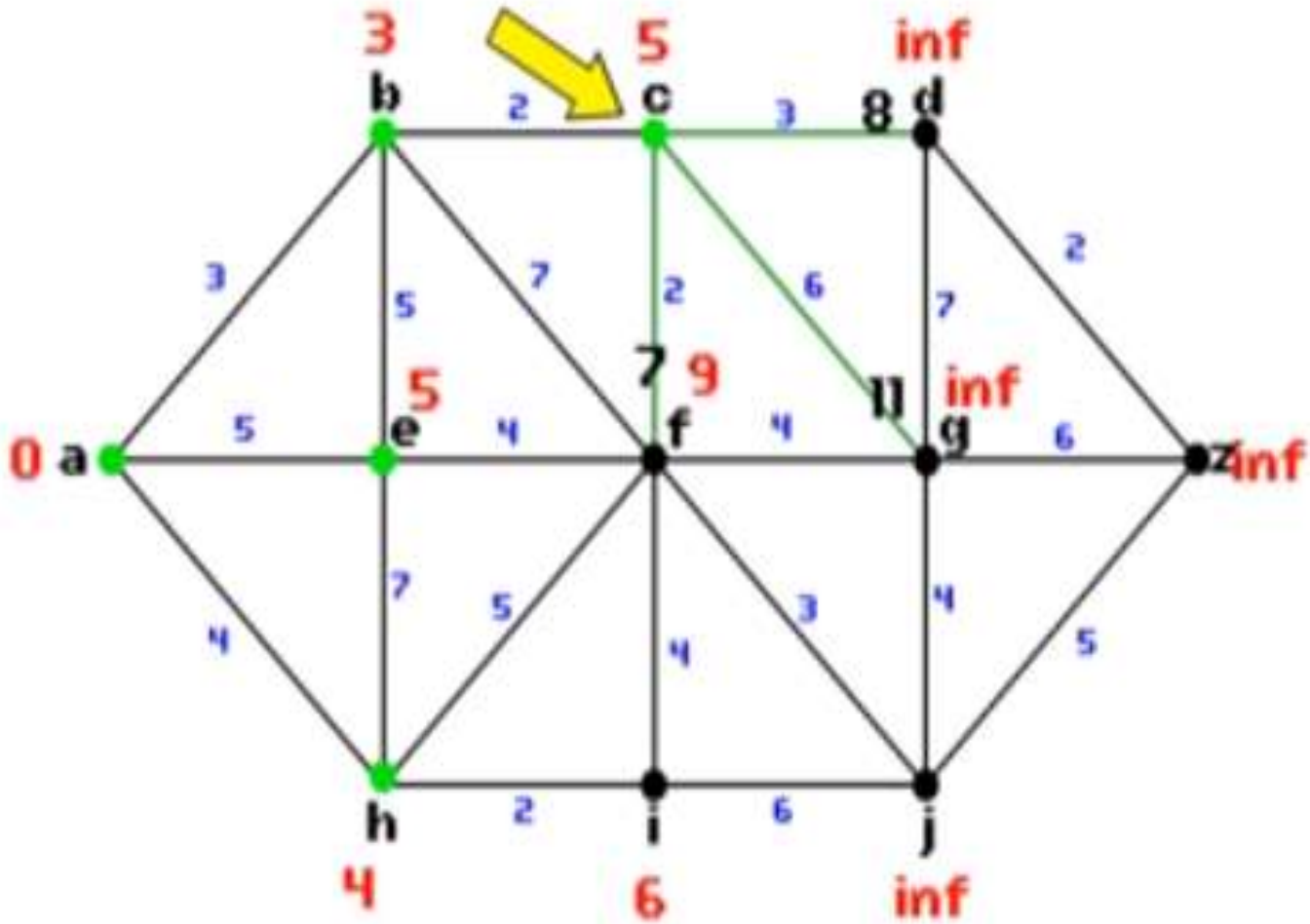
Repeat! Calculate distances.



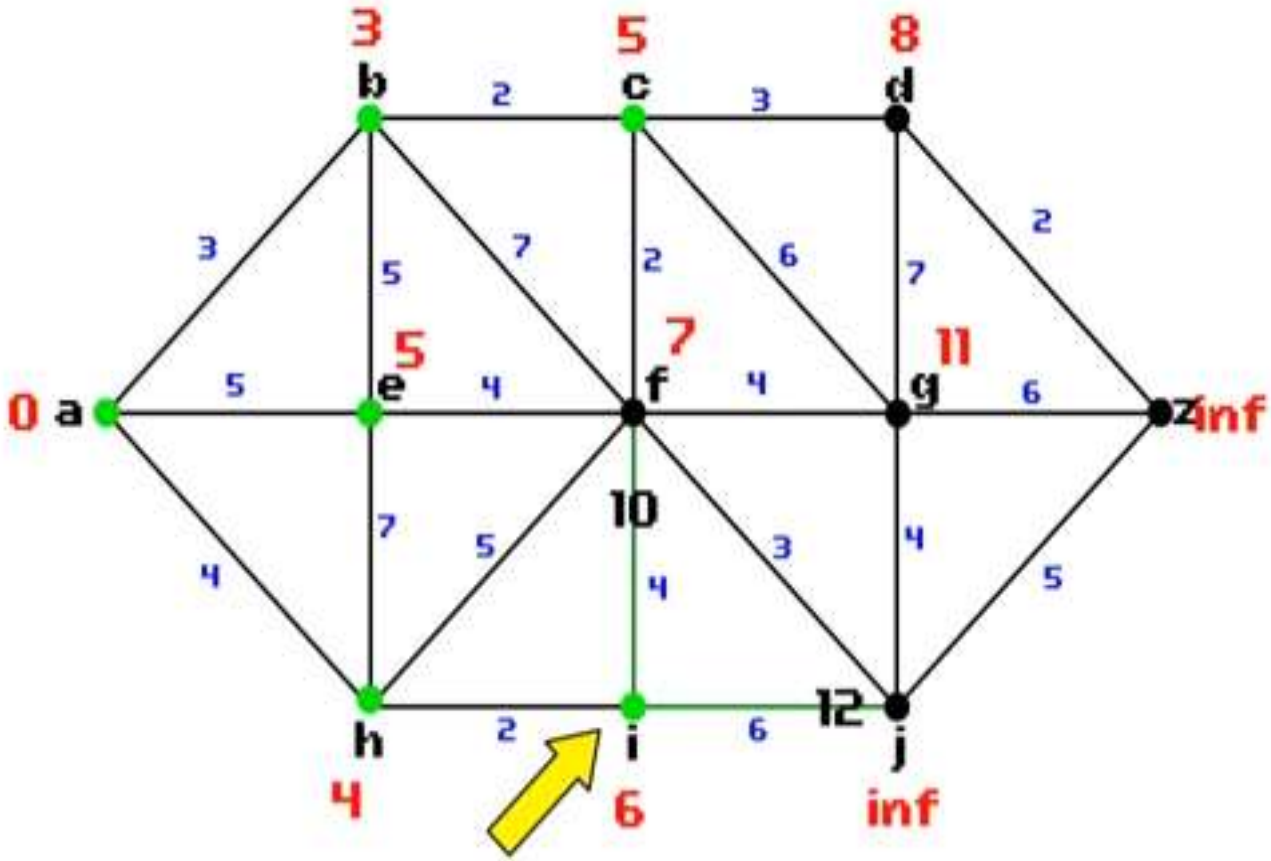
Removing from graph...



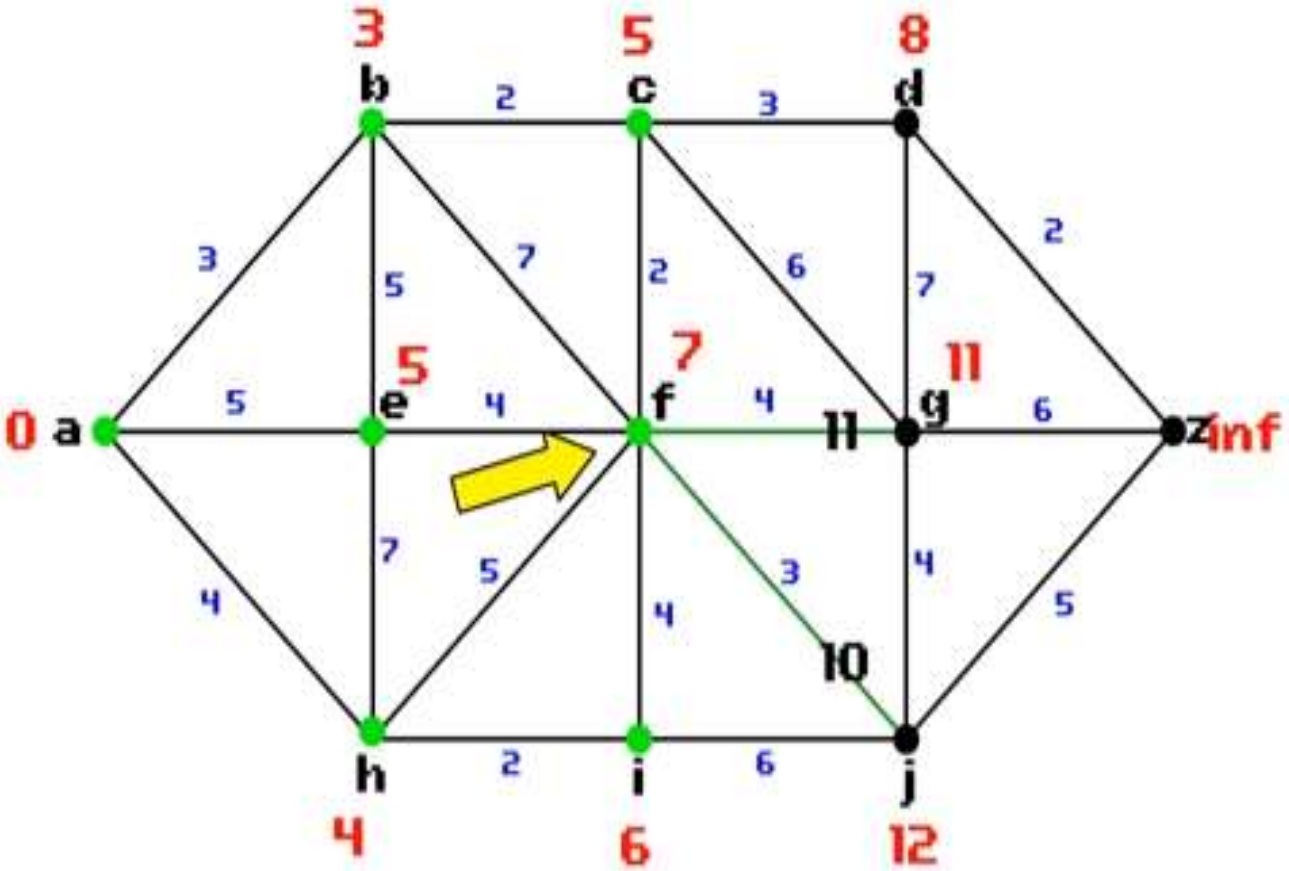
And so on...



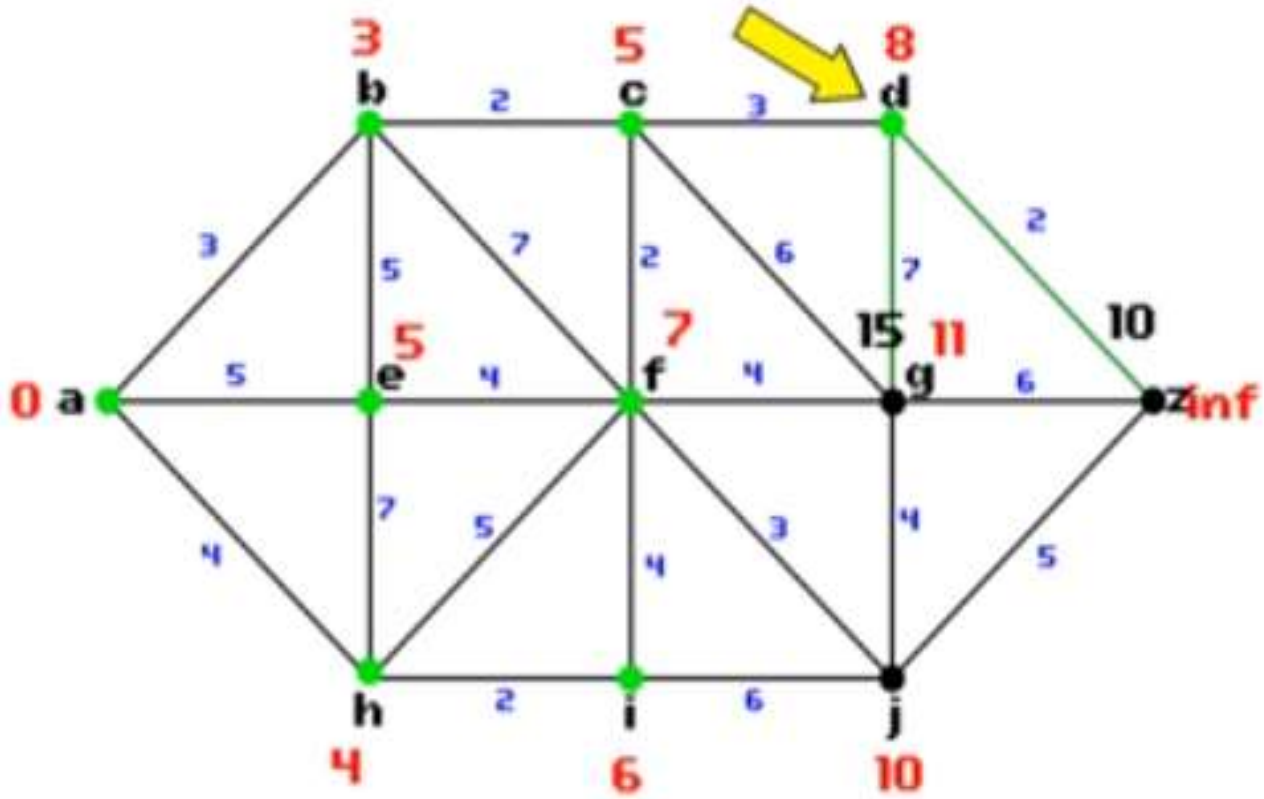
And so on...


3
b
5
c
8
d
2
3
3
5
7
2
6
7
2
0 a
5
e
5
4
f
7
4
g
11
6
z
inf
7
5
10
3
4
4
4
5
h
4
2
i
6
6
12
j
inf

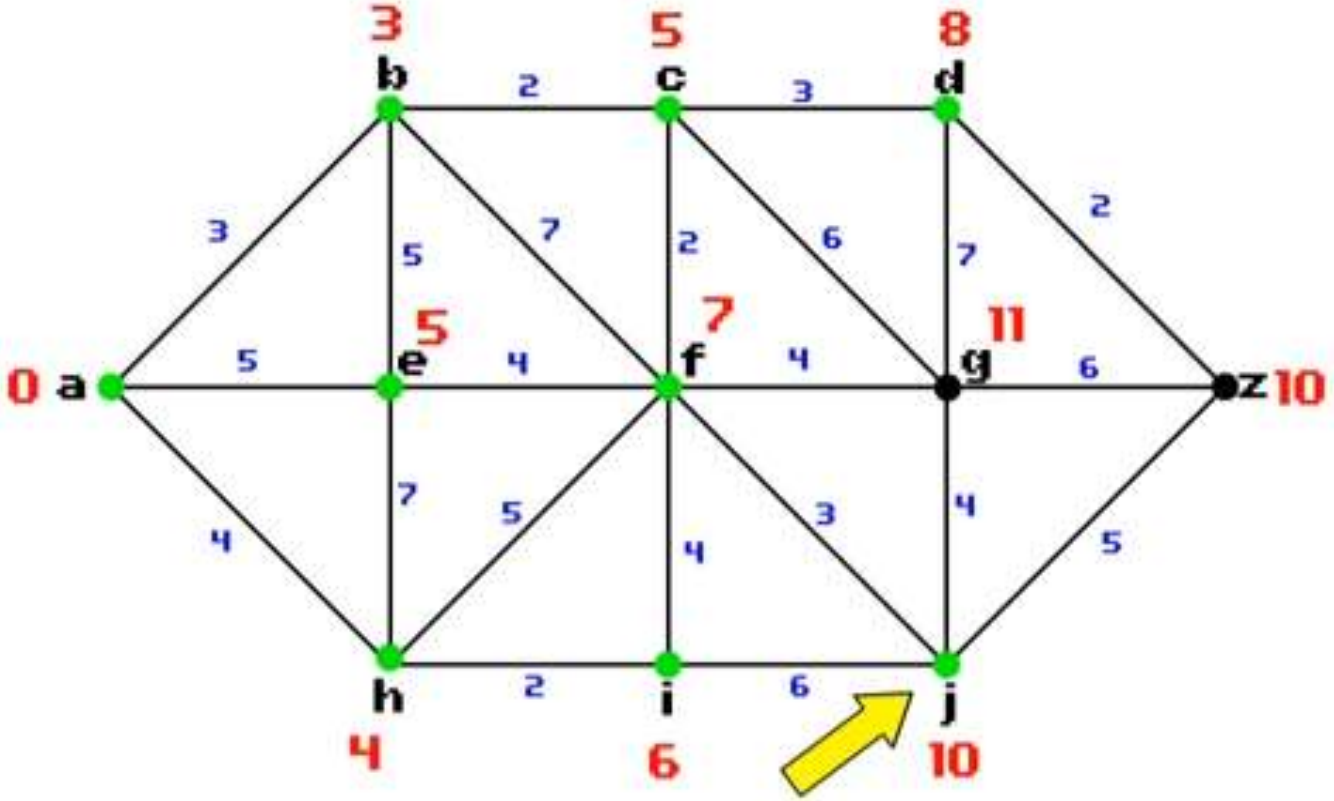
And so on...


3
b
2
5
c
3
8
d
3
5
7
2
6
7
2
0 a
5
5
e
4
7
f
4
11
11
g
6
z
inf
7
5
3
4
4
4
5
10
h
2
i
6
j
4
6
12

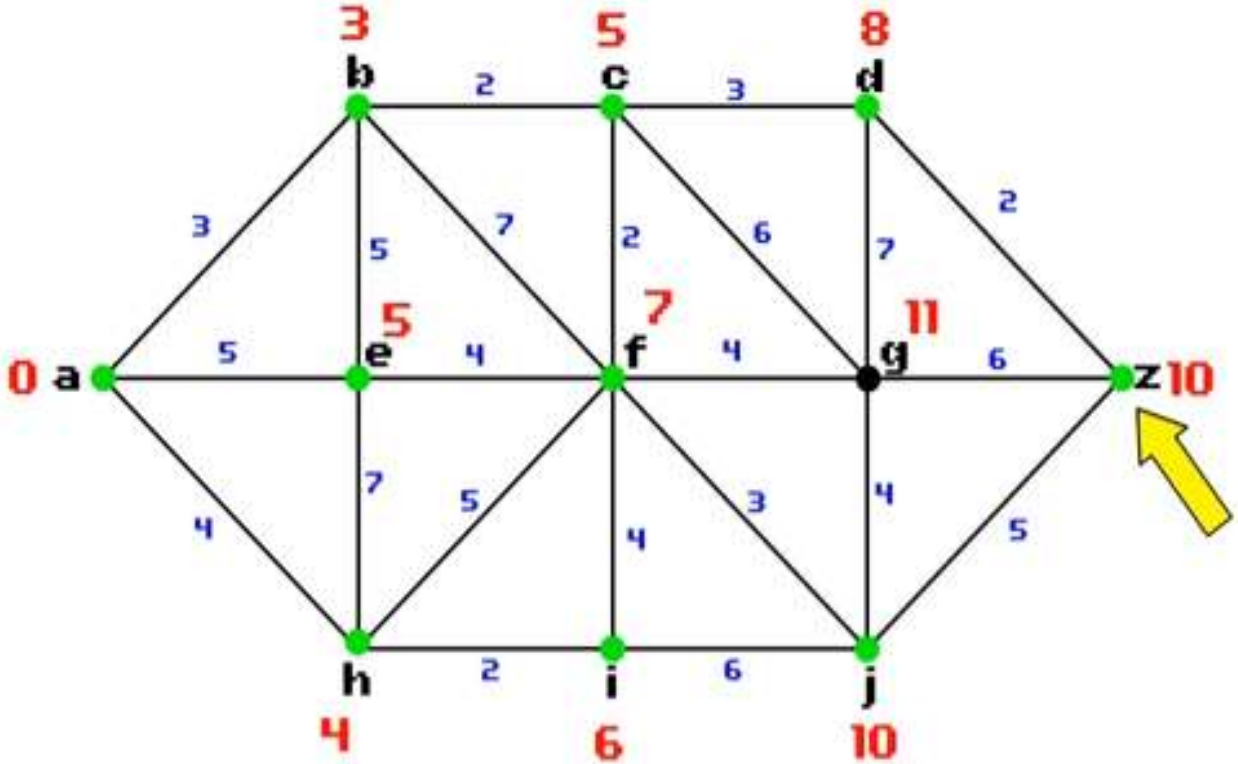
And so on...


3
b
5
c
8
d
2
3
3
5
7
2
6
7
2
0 a
5
e
5
4
f
7
4
15
g
11
6
z
10
inf
4
7
5
4
3
4
5
h
4
2
i
6
6
j
10

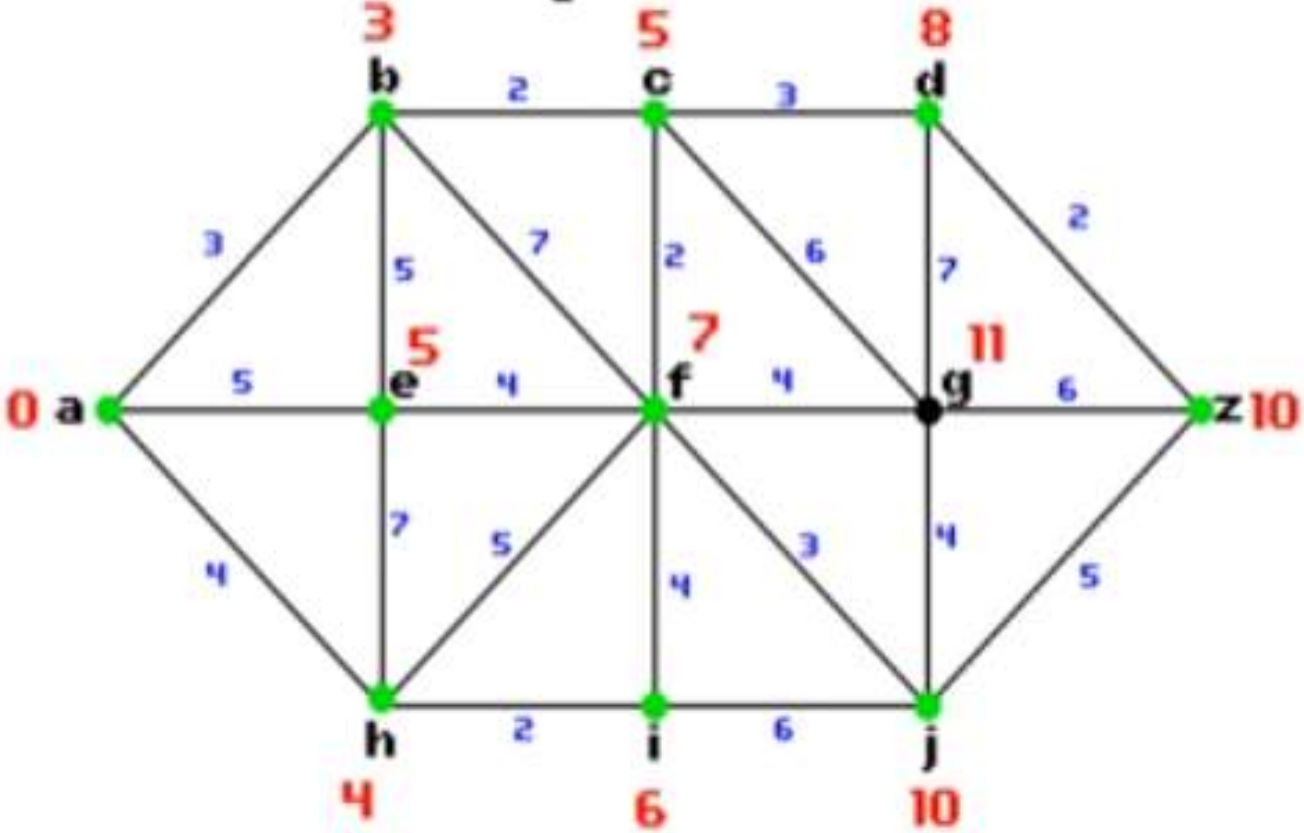
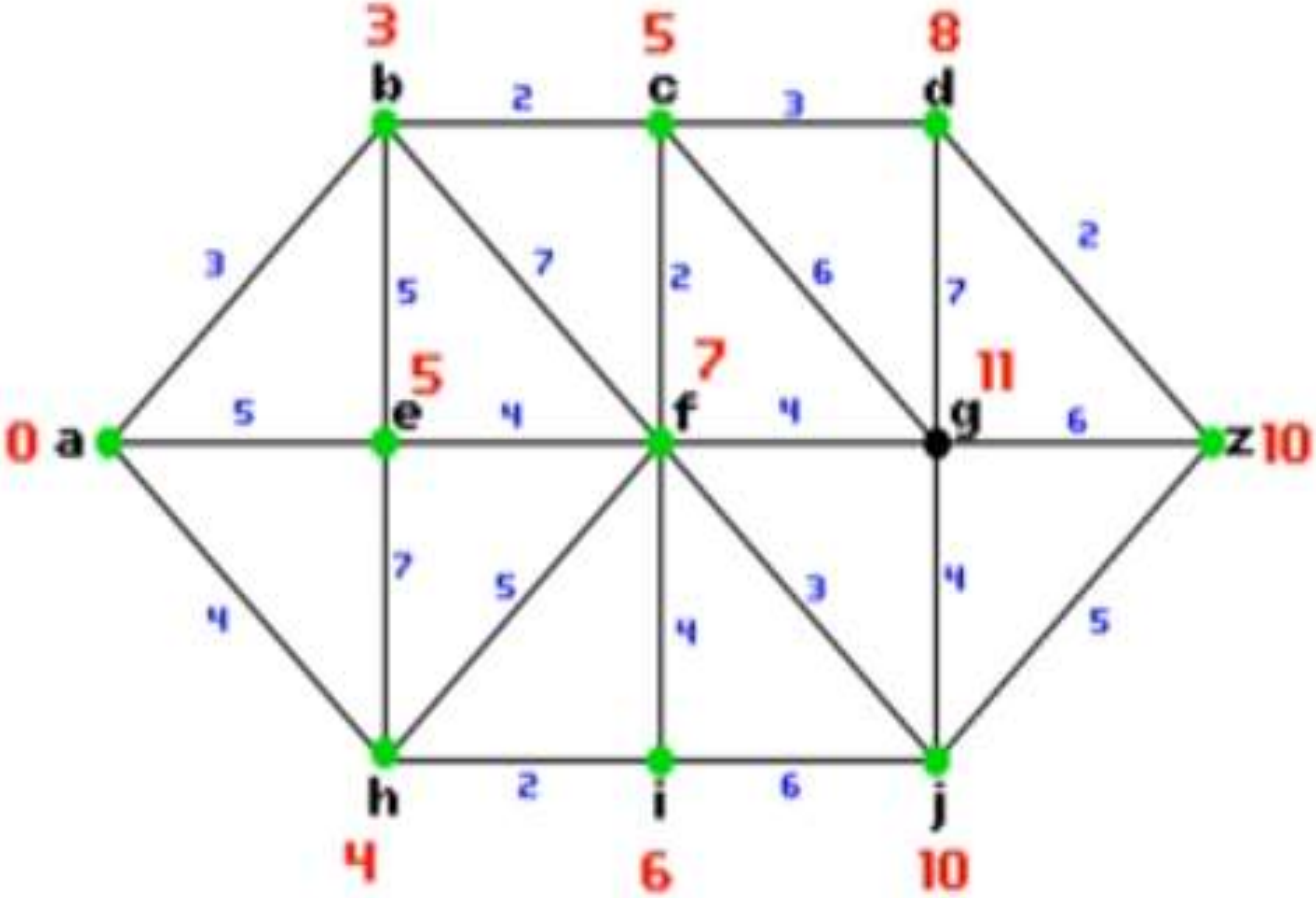
And so on...



And so on...



If the destination vertex is removed, the algorithm ends.



The least distance between a and z is 10.